

Entregable Semana 6

Ajxup Yax Elisa Yanira - 1553222
Castro Ordoñez Karol Mariánn - 1545522
Menchú Calderón Wilder Lionel - 1518222
Marroquin Escobar Juan Diego - 1689821
Hernandez De León Anderson Omar Noé - 1512321
Quemé Cárcamo Jesús Marco Antonio - 1592920
Bustamante Orozco Guillermo Enrique - 1584515
Tzunún Gutiérrez Estuardo Alexander - 1605922

Arquitectura General del Sistema

La arquitectura de Ecopac Digital se ha diseñado bajo una premisa fundamental: la sostenibilidad operativa y financiera para una organización sin fines de lucro. El sistema se desglosa en un front-end dual (Web y Móvil) que consume servicios centralizados a través de Supabase, garantizando la integridad de los datos y la simplicidad en el mantenimiento.

El stack tecnológico seleccionado responde a la necesidad de costo cero permanente y madurez técnica:

- React con Vite y Bootstrap (Web): Utilizado para la gestión administrativa, permitiendo interfaces reactivas y componentes reutilizables para el manejo de inventarios y reportes.
- React Native con Expo (Móvil): Crucial para la operación en comunidades rurales, facilitando el uso del sistema.
- Supabase (BaaS): Centraliza la autenticación, base de datos y almacenamiento de archivos en una sola plataforma, reduciendo la complejidad de infraestructura.
- PostgreSQL (Base de Datos): Garantiza la consistencia transaccional necesaria para el control estricto de lotes de medicamentos y trazabilidad médica.

Selección y Justificación de Tecnologías

Tras evaluar distintas alternativas, el equipo definió la siguiente solución: la aplicación web se desarrolla con React y se aloja en Vercel, la capa de servidor y la base de datos se concentran en Supabase, y la aplicación móvil se construye con React Native y se distribuye directamente al personal.

La información sobre planes y límites que aquí se cita fue verificada en agosto de 2026 contra la documentación oficial de cada plataforma, y conviene confirmar nuevamente antes del despliegue definitivo porque estas condiciones cambian con cierta frecuencia.

Criterios que guiaron la selección

Antes de justificar cada tecnología conviene explicar bajo qué reglas se tomaron las decisiones, porque varias de ellas se entienden sólo a la luz de estos criterios.

- Costo cero permanente: Este fue el criterio no negociable. Ecopac no puede pagar

mensualidades de infraestructura, ni ahora ni cuando el equipo entregue el proyecto.

- Simplicidad operativa después de la entrega: Mientras menos servicios distintos haya que mantener, mayor es la probabilidad de que el sistema siga funcionando cuando el equipo ya no esté acompañando a la organización. Este criterio terminó siendo determinante en la decisión final.
- Madurez y permanencia: Se descartaron herramientas experimentales. Ecopac deberá operar este sistema durante años después de la entrega, posiblemente sin apoyo técnico especializado.
- Posibilidad de crecer sin rehacer el trabajo: Cada componente debe poder trasladarse a infraestructura de pago o a créditos donados para organizaciones sin fines de lucro cambiando de proveedor, nunca de tecnología.

Tecnologías seleccionadas

Aplicación web: React con Vite y Bootstrap

La aplicación web está dirigida al trabajo administrativo: gestión de inventario, planificación de jornadas, administración de usuarios, generación de reportes y consulta de indicadores. React se eligió por su modelo de componentes reutilizables, que se ajusta bien a interfaces con muchos elementos repetitivos, como listados de medicamentos, tableros de jornadas y tablas de movimientos.

Vite acompaña a React como herramienta de desarrollo. Su aporte es de productividad, ya que reduce notablemente los tiempos de espera al programar, lo que importa cuando el equipo debe iterar con rapidez entre las reuniones de validación con la organización. Bootstrap se incorporó para resolver el diseño visual, pues proporciona componentes construidos y adaptables a distintos tamaños de pantalla sin exigir que el equipo invierta tiempo en diseño gráfico. Es una decisión razonable considerando que se trata de una herramienta administrativa interna, donde la prioridad es la claridad y la rapidez de uso.

Aplicación móvil: React Native con Expo

React Native permite que la aplicación móvil comparta el mismo lenguaje de programación que la aplicación web. En términos prácticos, la lógica de los formularios, las validaciones y

la comunicación con el servidor se escriben una vez y se aprovechan en ambas plataformas, en lugar de programarse dos veces. Esto responde exactamente a lo que el equipo planteó como necesidad de trasladar el trabajo de la web hacia la aplicación.

Expo complementa a React Native facilitando el acceso a las capacidades del teléfono, como la cámara para fotografías y el almacenamiento interno del dispositivo, sin necesidad de manipular configuraciones nativas de Android. Además permite guardar información dentro del propio teléfono. Expo también permite generar el archivo instalable de Android de forma gratuita, evitando los costos de las tiendas de aplicaciones.

Capa de servidor: Supabase

Supabase concentra en una sola plataforma los servicios que normalmente exigirían construirse por separado o contratarse con distintos proveedores. Sobre una base de datos PostgreSQL, agrega automáticamente una interfaz de programación que permite consultar y modificar la información desde la aplicación web y desde la aplicación móvil, un sistema de autenticación de usuarios con manejo de sesiones, un espacio de almacenamiento para archivos, un mecanismo de permisos definido en la propia base de datos y la capacidad de ejecutar rutinas propias para la lógica que el sistema requiera.

Varias de esas capacidades responden de forma directa a los requerimientos levantados con la organización. El mecanismo de permisos permite establecer que el personal médico acceda únicamente a fichas de pacientes y registro de consultas, mientras la administración conserva acceso total. Como esas reglas se definen en la base de datos y no en cada aplicación, se aplican siempre, sin importar si la consulta proviene de la web o del teléfono, lo cual refuerza la protección de los datos clínicos exigida en los requerimientos no funcionales.

El sistema de autenticación cubre el inicio y cierre de sesión mediante credenciales y la asignación de roles, evitando que el equipo tenga que construir desde cero un componente delicado en materia de seguridad. El almacenamiento de archivos, separado de la base de datos, deja resuelto el escenario de conservar fotografías de las fichas clínicas físicas o imágenes de medicamentos sin necesidad de contratar otro servicio. Las rutinas programadas permiten atender el requerimiento de generar alertas automáticas un mes antes del vencimiento de los medicamentos, y la capacidad de reflejar cambios en tiempo real favorece

que la consulta de existencias durante una jornada muestre siempre la disponibilidad actualizada.

Base de datos: PostgreSQL

La base de datos que Supabase administra es PostgreSQL, y esa fue una razón de peso para inclinarse por esta plataforma. PostgreSQL destaca por su solidez en el manejo de transacciones. En un sistema que controla existencias de medicamentos, es indispensable que una entrega quede registrada por completo o no quede registrada en absoluto. Un registro a medias produciría diferencias entre lo que el sistema reporta y lo que realmente hay en bodega, que es precisamente el problema que la organización busca resolver.

Esa característica es la que permite cumplir el requerimiento de que todo movimiento de inventario permanezca en estado pendiente de validación hasta que la administradora lo apruebe, y solo entonces afecte las existencias oficiales, sin que puedan producirse estados intermedios que dejen el inventario descuadrado.

Su naturaleza relacional también se ajusta a la forma de los datos del proyecto. Pacientes, consultas, jornadas, lotes de medicamentos y voluntarios se relacionan entre sí de manera estructurada, y ese tipo de vínculos es lo que un motor relacional maneja de forma natural. Un aspecto adicional a favor es que PostgreSQL es un estándar ampliamente soportado, de modo que la información puede trasladarse a otro proveedor en el futuro sin quedar atada a Supabase.

Panorama de opciones gratuitas evaluadas

Antes de definir la solución se revisó el panorama actual de plataformas que permiten desplegar sin costo. Es un terreno que se ha ido reduciendo con los años: Heroku eliminó su plan gratuito y ElephantSQL, que durante mucho tiempo fue una opción común para bases PostgreSQL, cerró operaciones por completo. Lo que queda hoy puede agruparse en tres categorías, y entender a cuál pertenece cada proveedor importa más que su lista de funciones, porque esa categoría anticipa cómo y cuándo termina la gratuidad.

- Plataformas con crédito de prueba: Railway funciona bajo este esquema, entregando

un crédito inicial por única vez. Se descartó porque una vez agotado ese crédito el servicio se detiene, lo que convierte la gratuidad en algo temporal y no en una solución sostenible.

- Plataformas con plan gratuito que expira: Aquí se ubica la base de datos PostgreSQL gratuita de Render, que resulta particularmente riesgosa, expira treinta días después de su creación y, tras un periodo de gracia de catorce días, la plataforma la elimina junto con toda la información almacenada. Para un sistema que guardará historiales clínicos, un borrado automático a los cuarenta y cuatro días es inaceptable, por lo que quedó descartada de inmediato.
- Plataformas con plan gratuito permanente: Es la categoría donde se ubica Supabase, la opción seleccionada. Dentro de esta misma categoría se evaluó seriamente Aiven, que ofrece un gigabyte de almacenamiento e incluye respaldos automáticos, ventaja real frente a Supabase. Sin embargo, Aiven proporciona únicamente la base de datos, de modo que habría exigido administrar además un servicio separado para la capa de servidor, elevando el sistema a tres plataformas. También se revisó Neon, que ofrece PostgreSQL con reactivación muy rápida pero apenas medio gigabyte de almacenamiento y límites mensuales de procesamiento que, al agotarse, suspenden la base hasta el siguiente ciclo.

En cuanto al alojamiento de la aplicación web, además de Vercel se revisaron Cloudflare Pages, que ofrece ancho de banda sin límite y permite uso comercial de forma explícita, y Netlify, que migró hacia un modelo de créditos mensuales menos predecible. Vercel se seleccionó por su integración directa con el repositorio de código y su facilidad de configuración, mientras que Cloudflare Pages queda identificada como alternativa de respaldo por una razón que se explica más adelante.

Despliegue

El despliegue se concentra en dos plataformas. La aplicación web se aloja en Vercel, mientras que la capa de servidor y la base de datos residen en Supabase. La aplicación móvil se compila mediante los servicios de Expo y se distribuye directamente a los teléfonos del personal, sin intermediarios.

Aplicación web en Vercel

Vercel aloja la aplicación web bajo su plan gratuito, denominado Hobby. Incluye certificado de seguridad y conexión cifrada de forma automática, lo que satisface el requerimiento de que toda comunicación entre el cliente y el servidor viaje cifrada. Incorpora además despliegue continuo conectado directamente al repositorio de GitHub, de modo que cada actualización del código se publica sin intervención manual, y distribuye el contenido desde servidores ubicados en distintas regiones, lo que mejora los tiempos de carga.

Sus límites mensuales son holgados para el volumen que manejará Ecopac: alrededor de cien gigabytes de transferencia de datos, hasta un millón de solicitudes y seis mil minutos de compilación. Estos contadores se reinician cada treinta días. Si alguno se agotara, el servicio afectado se detiene hasta el siguiente periodo, sin cobros automáticos, ya que el plan no requiere registrar tarjeta de crédito. Considerando que se prevén alrededor de quince usuarios, el consumo real quedará muy por debajo de estos techos.

Conviene señalar con transparencia que los términos de Vercel restringen el plan Hobby a proyectos personales y no comerciales, con una definición amplia de uso comercial que abarca cualquier despliegue que genere beneficio económico para alguien involucrado en su producción. El caso de Ecopac se ubica cómodamente fuera de esa definición, ya que el sistema lo desarrolla un equipo estudiantil sin remuneración y el resultado es una herramienta interna que no vende nada ni muestra publicidad. Aun así, se trata de una zona interpretativa y no de una excepción garantizada por escrito.

Capa de servidor y base de datos en Supabase

El plan gratuito de Supabase contempla quinientos megabytes de base de datos, un gigabyte adicional para almacenamiento de archivos, cinco gigabytes de transferencia mensual, hasta cincuenta mil usuarios activos al mes en el sistema de autenticación, quinientas mil ejecuciones mensuales de rutinas propias y hasta dos proyectos activos. Permite uso comercial de manera explícita, no impone fecha de vencimiento y no solicita tarjeta de crédito.

Dimensionando esas cifras contra la realidad del proyecto, el margen es amplio. La

organización atiende un promedio de cincuenta pacientes por jornada y prevé alrededor de quince usuarios del sistema, cifra muy distante del límite de cincuenta mil usuarios activos. En cuanto al almacenamiento, quinientos megabytes alcanzan para varios años de registros de texto como pacientes, consultas, lotes y movimientos de inventario. El punto de atención está en no almacenar imágenes dentro de la base de datos, ya que el gigabyte destinado a archivos existe precisamente para ese fin y mantiene ambos consumos separados.

La plataforma también cifra la información en tránsito y en reposo, y permite definir qué usuarios acceden a qué datos mediante reglas aplicadas en la propia base, aspectos que responden a los requerimientos no funcionales relacionados con la confidencialidad de las fichas clínicas y la restricción del acceso a diagnósticos e historiales médicos.

Compilación y distribución de la aplicación móvil

La aplicación móvil se compila mediante el servicio de compilación en la nube de Expo, cuyo plan gratuito incluye quince compilaciones de Android al mes. Ese margen resulta suficiente tanto para el desarrollo como para publicar versiones nuevas al personal. El proyecto se enfoca exclusivamente en Android porque ese es el sistema que efectivamente utilizan los médicos y voluntarios en campo.

Para hacer llegar la aplicación a los teléfonos existen dos caminos, ambos sin costo. El primero consiste en entregar el archivo instalable directamente al personal mediante un enlace o por mensajería, lo que exige que cada usuario autorice una vez la instalación desde orígenes externos en su teléfono, paso que conviene incluir en la capacitación inicial.

Comportamiento de reposo por inactividad

El reposo por inactividad es la contrapartida que las plataformas gratuitas imponen y afecta de manera directa la experiencia del personal durante las jornadas, por lo que merece tratarse en detalle.

- Aplicación web en Vercel: no entra en reposo. El contenido se sirve desde una red de distribución y permanece disponible de forma permanente. Lo único que se reinicia cada treinta días son los contadores de consumo mensual.

- Supabase: reposa a los siete días. El proyecto se pausa cuando transcurren siete días consecutivos sin actividad sobre la base de datos. Los datos permanecen intactos, no se pierde nada. La reactivación se realiza desde el panel de administración y tarda alrededor de treinta segundos.

Riesgos y soluciones

- Pausa por inactividad: Se resuelve programando una tarea automática gratuita mediante las acciones programadas de GitHub, que ejecuta cada tres o cuatro días una consulta mínima contra la base de datos. Con eso el contador de inactividad se reinicia solo y el proyecto nunca llega a pausarse. Se configura una sola vez y funciona de manera desatendida, sin depender de que alguien recuerde hacerlo. Como complemento operativo, se recomienda que el personal abra la aplicación unos minutos antes de comenzar la atención a pacientes.
- Ausencia de respaldos automáticos: Este es el punto que exige mayor disciplina y la principal desventaja frente a otras alternativas evaluadas. El plan gratuito no incluye respaldos, por lo que deben programarse desde el inicio del despliegue y no dejarse como una tarea pendiente. Se resuelve con el mismo mecanismo de tareas automáticas, generando una copia periódica de la información y almacenándola en un repositorio privado o en el espacio institucional de la organización. Tratándose de historiales clínicos que no pueden reconstruirse si se pierden, se recomienda verificar periódicamente que esas copias efectivamente se estén generando.
- Lógica distribuida entre la aplicación y la base de datos. Al no existir una capa de servidor propia que centralice las reglas de negocio, estas quedan repartidas entre las reglas de permisos, las rutinas programadas y las aplicaciones cliente. Se recomienda documentar con claridad dónde reside cada regla, de modo que quien retome el mantenimiento del sistema en el futuro pueda ubicarlas sin dificultad.

Costos

La totalidad de la infraestructura opera sin costo alguno. El alojamiento de la aplicación web en Vercel, la capa de servidor y la base de datos en Supabase, la compilación y distribución de la aplicación móvil mediante Expo, y las tareas automáticas de mantenimiento y respaldo

mediante GitHub funcionan bajo planes gratuitos permanentes que no requieren registrar tarjeta de crédito. El total mensual es de cero dólares, y ninguna de estas plataformas exige un pago futuro para seguir operando.

Diagrama de Despliegue

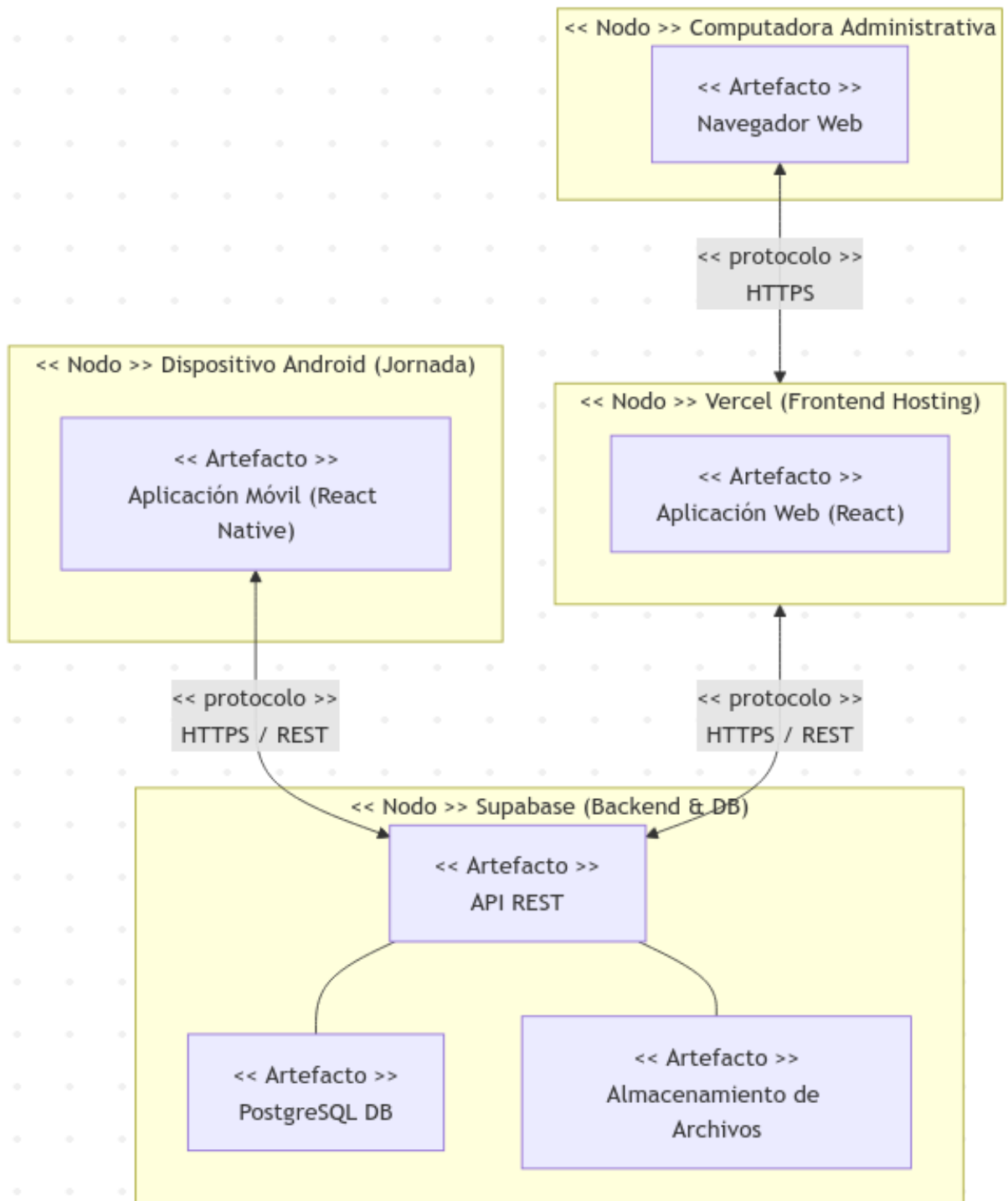
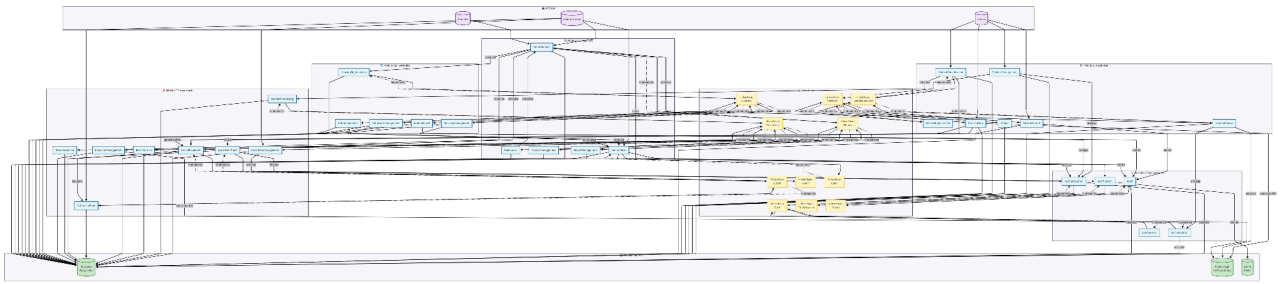
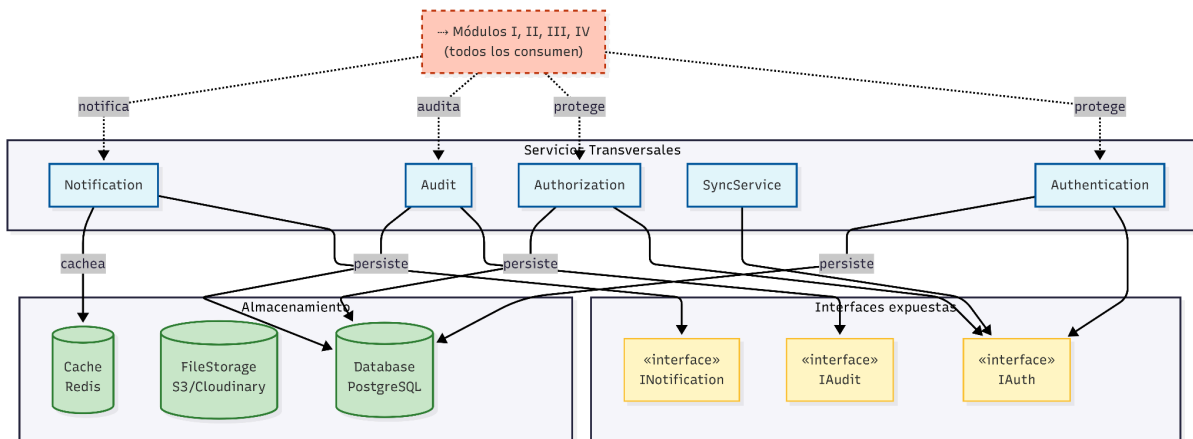


Diagrama de componentes para representar módulos y dependencias



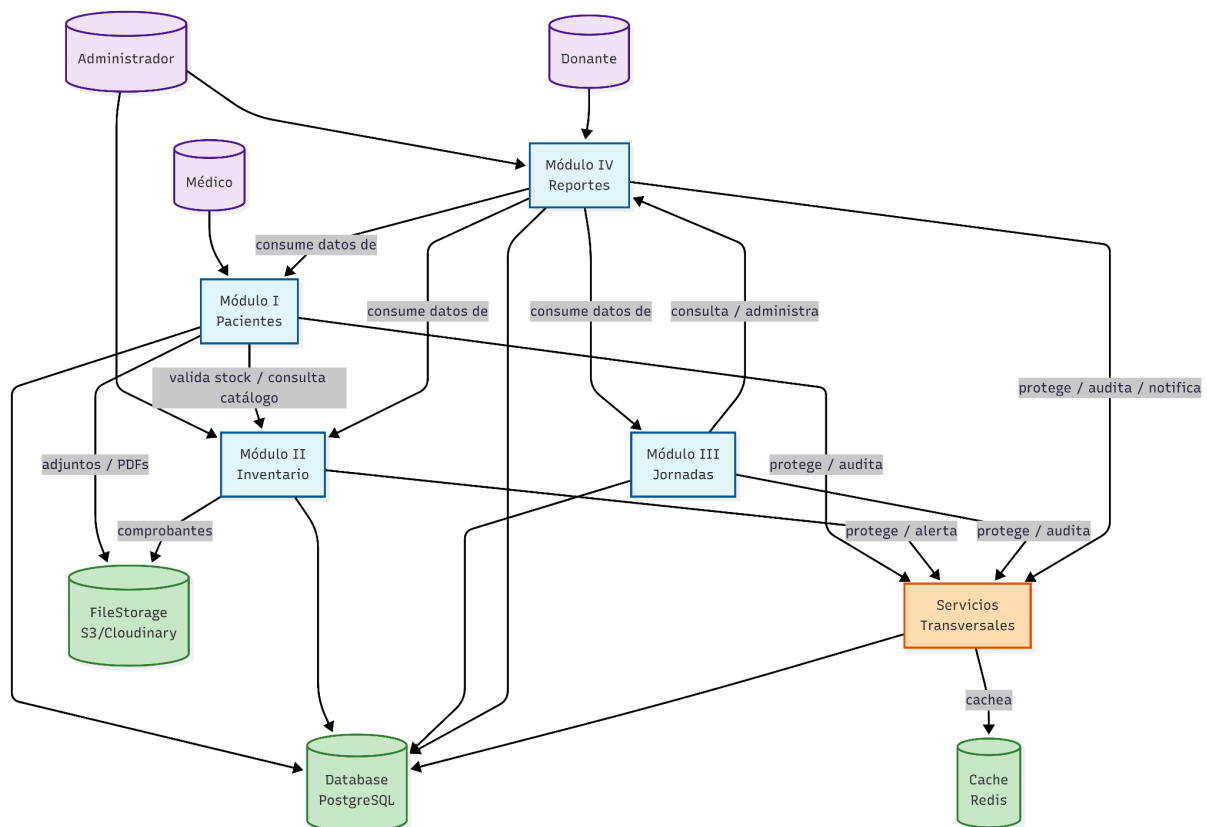
Vista General

Representa la arquitectura de componentes del sistema en su capa más alta de abstracción, mostrando cuatro módulos funcionales (Pacientes, Inventario, Jornadas, Reportes) como bounded contexts independientes que se comunican mediante dependencias unidireccionales explícitas, un servicio transversal centralizado (Authentication, Authorization, Audit, Notification) consumido por todos los módulos bajo un patrón de cross-cutting concerns, y una capa de persistencia poliglota compuesta por PostgreSQL como base de datos relacional principal, un object storage (S3/Cloudinary) para binarios, y Redis como cache in-memory. El diagrama evidencia que Reportes actúa como un agregador de solo lectura (read model) que consume datos de los otros tres módulos, sugiriendo un patrón cercano a CQRS a nivel de sistema.



Vista General #2

Ilustra la infraestructura de cross-cutting concerns consumida horizontalmente por los cuatro módulos de negocio, implementando Authentication, Authorization y SyncService bajo una interfaz común (IAuth) que sugiere un mecanismo de autenticación federado o compartido, mientras Notification y Audit exponen contratos independientes (INotification, IAudit) para desacoplar el registro de auditoría del envío de notificaciones. La persistencia diferenciada evidencia una estrategia de optimización de lecturas/escrituras según el patrón de acceso de cada servicio, reduciendo la carga sobre la base de datos relacional para operaciones de alta frecuencia y baja durabilidad como las notificaciones.

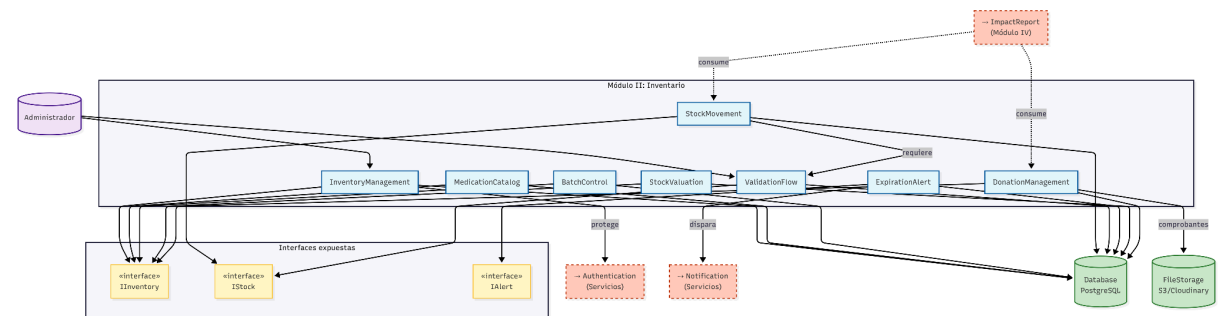


Modulo Pacientes

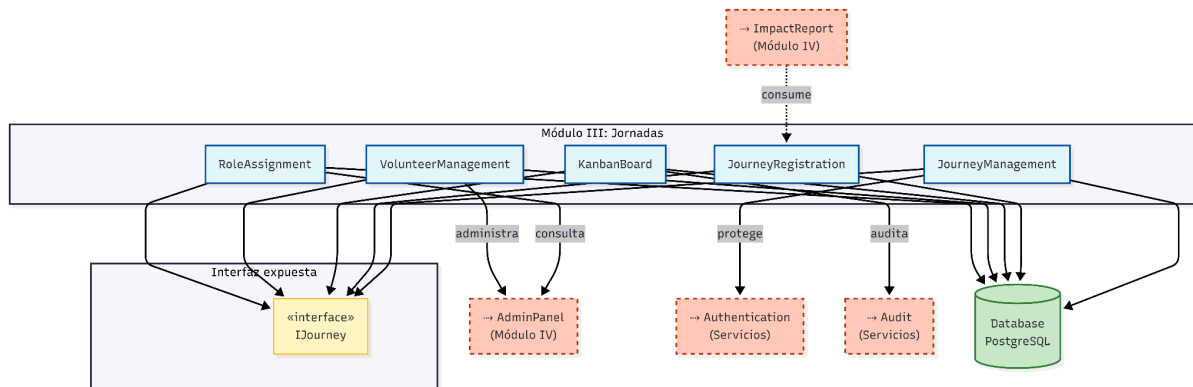
Modela el subsistema de gestión clínica, donde los componentes internos (PatientManagement, MedicalRecord, Triage, MedicalConsultation, Prescription, PatientSearch) exponen su contrato público a través de dos interfaces segregadas (IPatient e IMedicalRecord), aplicando el principio de segregación de interfaces (ISP) para desacoplar las operaciones de identificación/búsqueda de paciente de las operaciones sobre el expediente clínico. Las dependencias salientes hacia StockMovement, MedicationCatalog y

ExpirationAlert del Módulo II reflejan un acoplamiento funcional necesario para validar existencias antes de emitir una prescripción, mientras que la persistencia dual (Database para datos estructurados y FileStorage para adjuntos/PDFs) indica una separación entre almacenamiento transaccional y almacenamiento de objetos no estructurados.

Módulo Inventario

Módulo Jornada

ImpactReport como consumidor de solo lectura, consolidando el patrón de agregación de reportes observado en el sistema completo.



Módulo de Reportes

Constituye la capa de agregación y presentación (reporting/analytics layer) del sistema, donde ImpactReport funge como orquestador central que consulta de forma cruzada a MedicalRecord, StockMovement, DonationManagement y JourneyRegistration de los otros tres módulos, configurando un patrón de composición tipo facade o aggregator sobre múltiples bounded contexts. Dashboard consume a su vez a ImpactReport en lugar de acceder directamente a las fuentes primarias, respetando una arquitectura en capas (layered architecture) que evita el acceso directo de la capa de presentación a los datos crudos; AdminPanel, por su parte, expone IUser como interfaz administrativa independiente de IReport, separando las responsabilidades de gestión de usuarios/roles de las de generación de reportes.

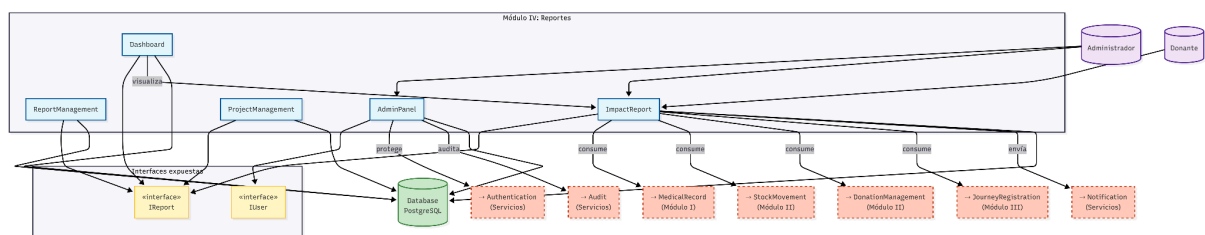
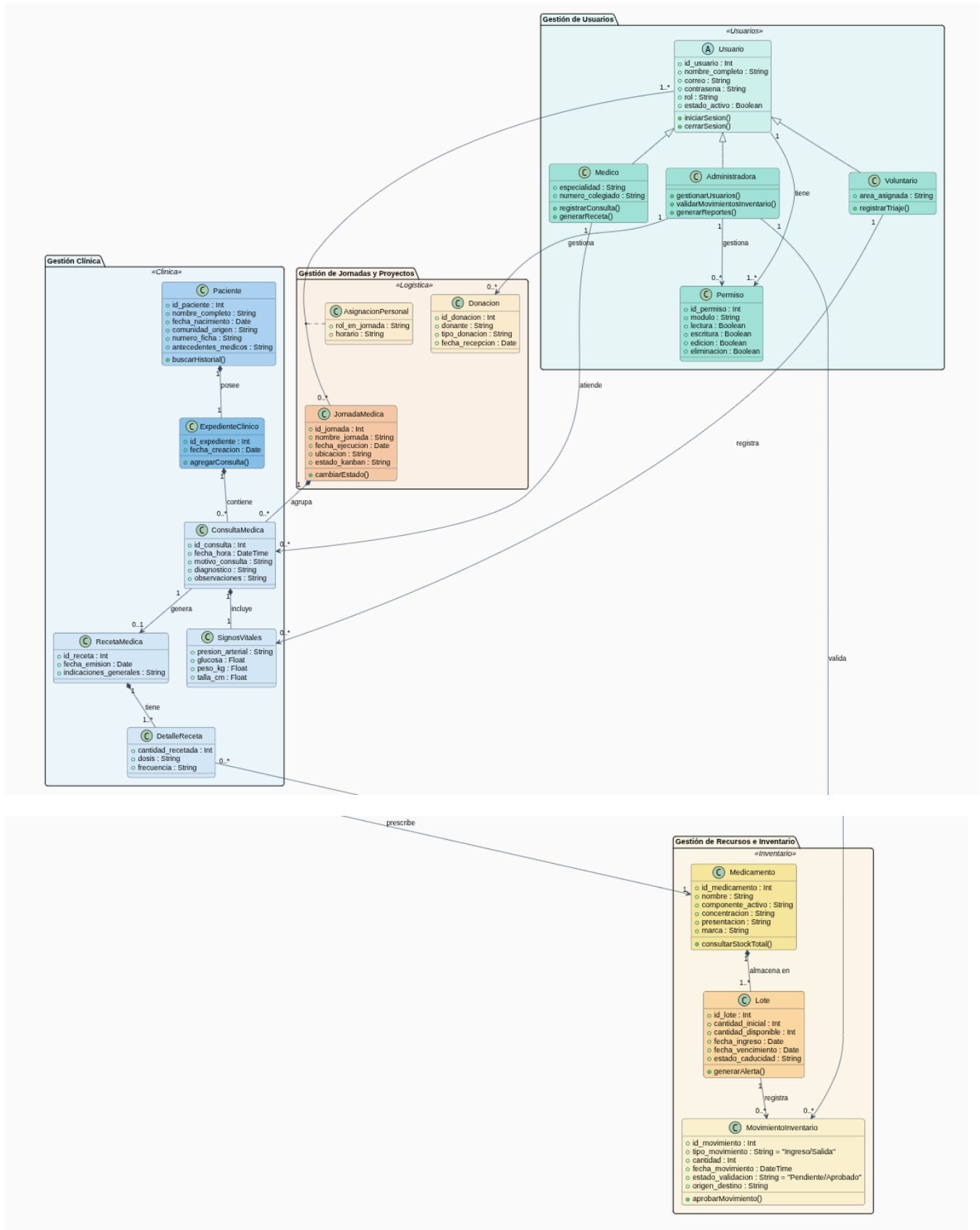


Diagrama UML de Clases



Diseño de la Base de Datos

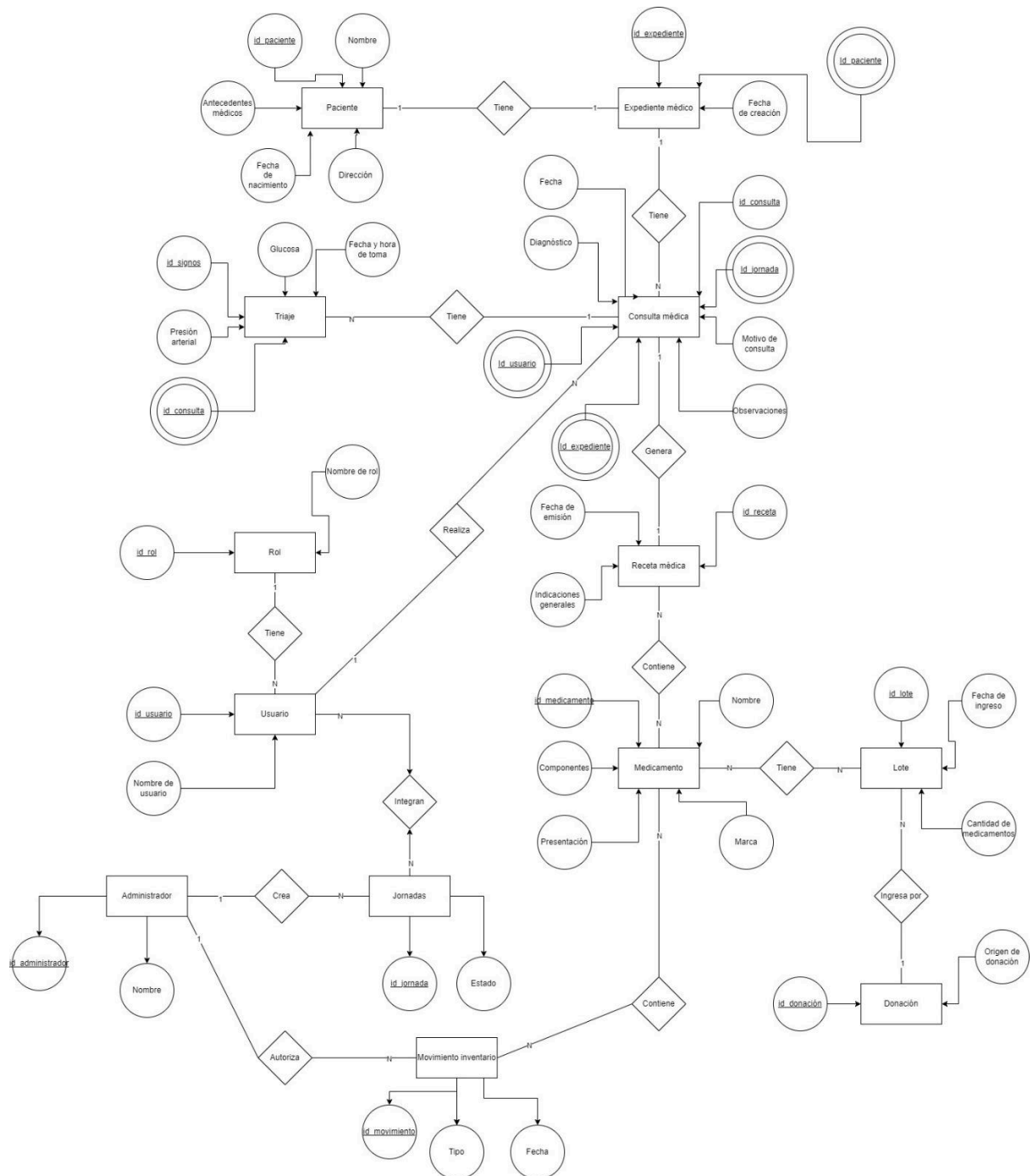
Se desarrolla en tres niveles de abstracción: el modelo conceptual que identifica las entidades del dominio y sus relaciones sin considerar aspectos técnicos, el modelo lógico que detalla atributos, llaves y cardinalidades y el modelo físico que traduce todo lo anteriormente mencionado a estructuras de PostgreSQL sobre la plataforma Supabase.

Convenciones de nomenclatura

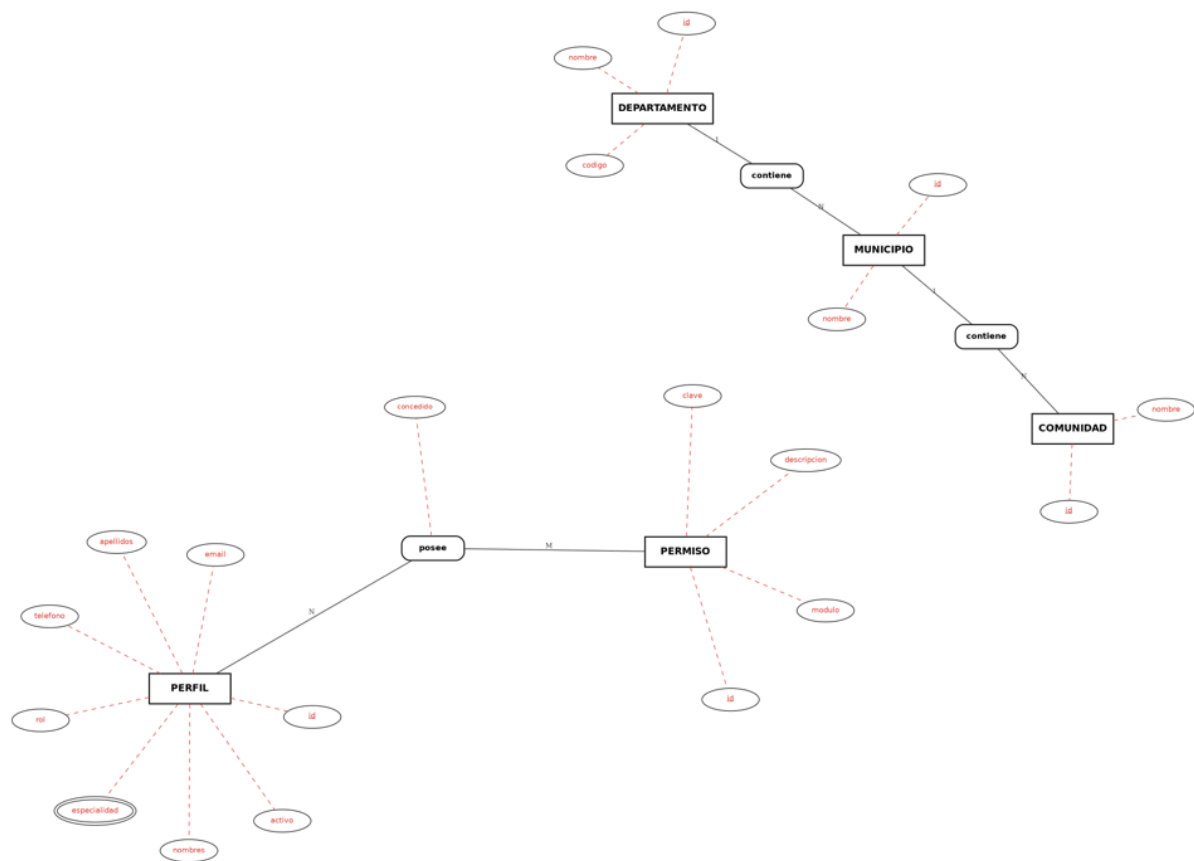
Se aplicaron de forma uniforme para que el esquema sea predecible y el resto del equipo pueda extenderlo sin ambigüedades.

- Nombres en español, en minúsculas y con guión bajo como separador.
- Tablas en plural y columnas en singular.
- La llave primaria siempre se llama id.
- Las llaves foráneas usan el nombre de la tabla referida en singular con el sufijo _id.
- Toda tabla incluye created_at; las que se modifican incluyen también updated_at.
- Los índices llevan el prefijo idx_, las restricciones únicas uq_ y las de verificación chk_.

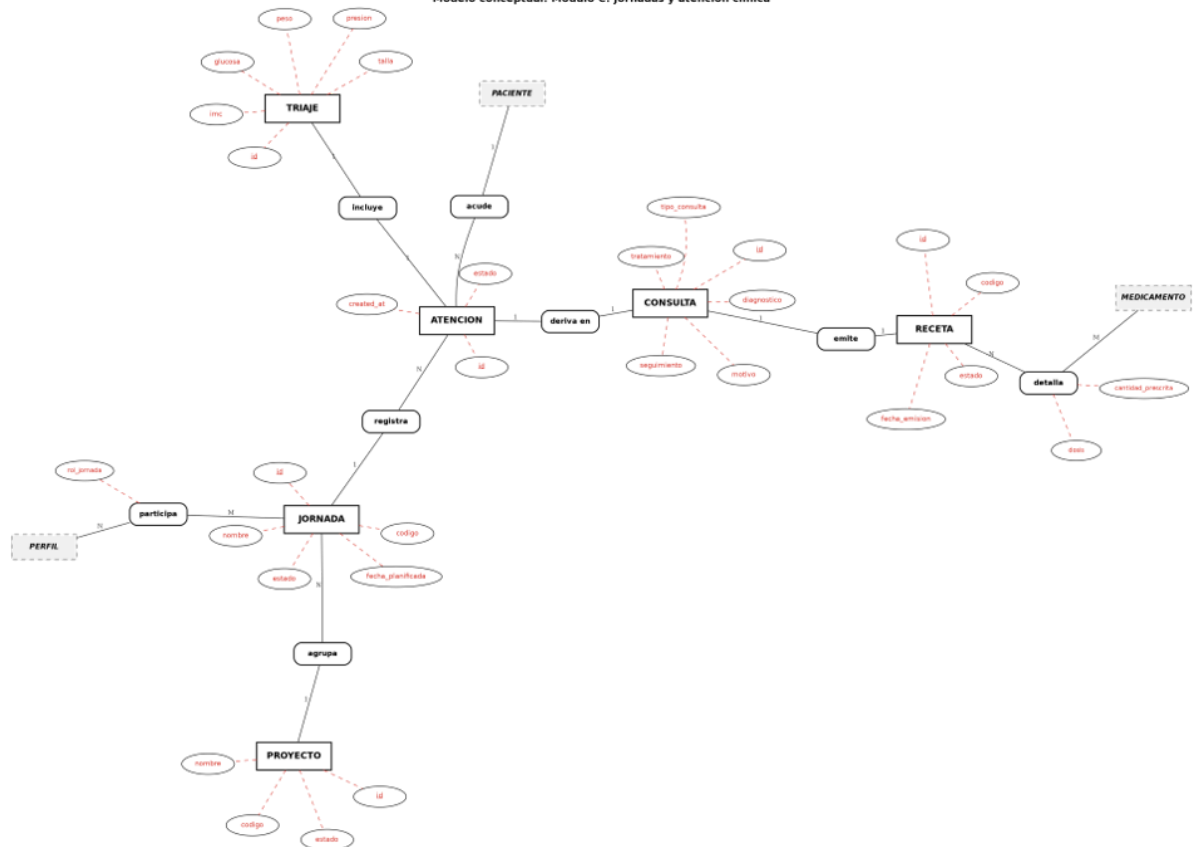
Modelo Conceptual



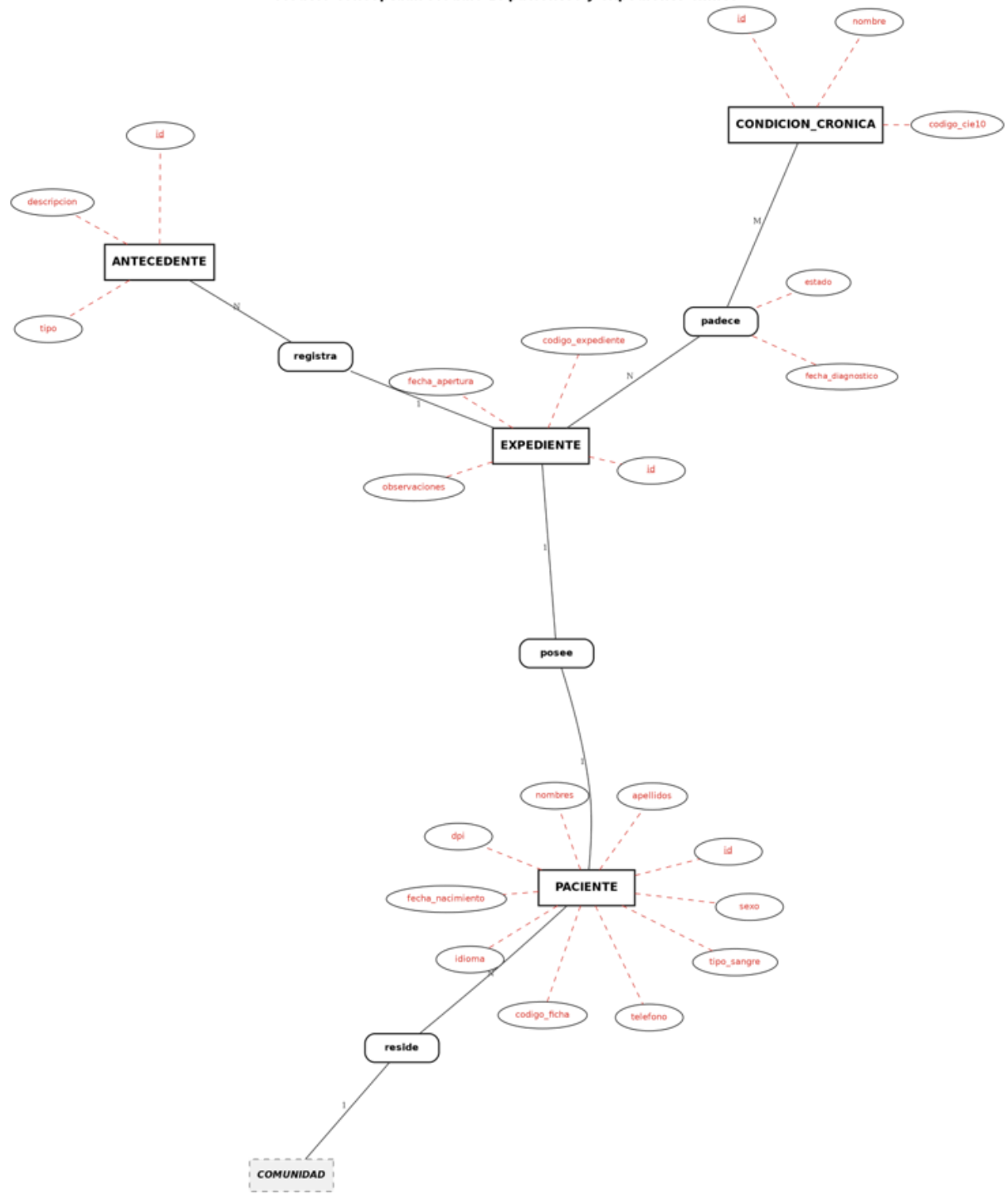
Modelo Lógico



Modelo conceptual. Módulo C: jornadas y atención clínica



Modelo conceptual. Módulo B: pacientes y expediente clínico



Entidades Principales

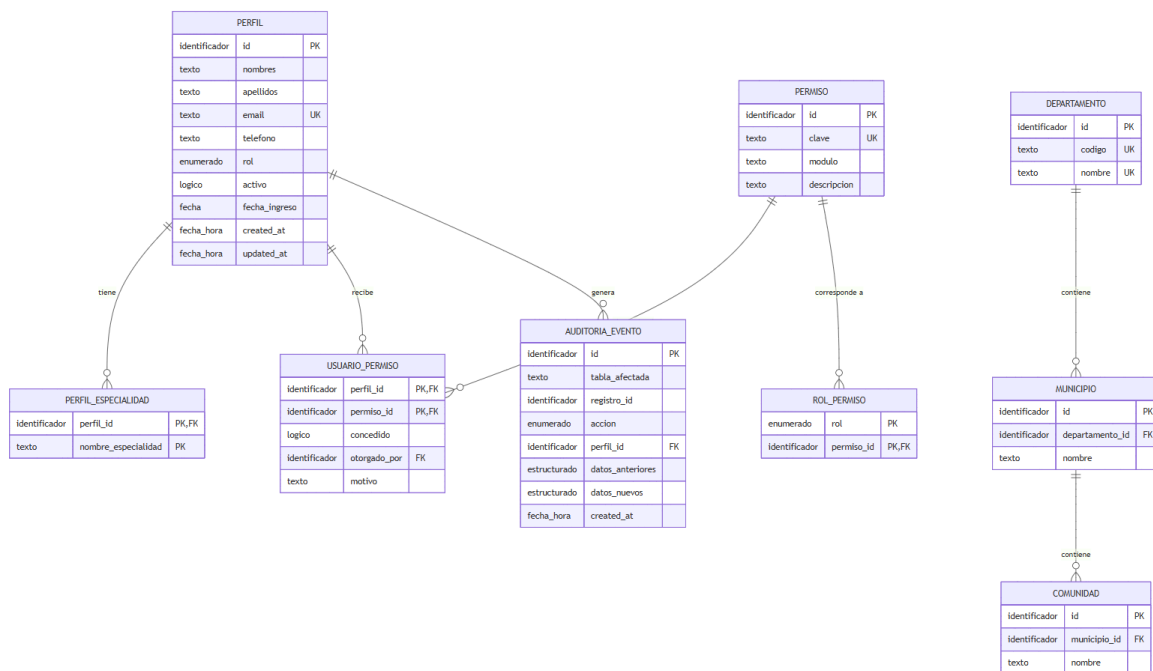
Entidad	Descripción
Usuario	Persona con acceso al sistema: administración, junta directiva, socios, médicos y voluntarios.
Comunidad	Localidad donde reside un paciente o se realiza una jornada. Se organiza en municipios y departamentos.
Paciente	Beneficiario atendido. Se identifica por número de ficha, nombre y comunidad, ya que muchos carecen de documento.
Expediente	Historia clínica única del paciente. Acumula antecedentes, condiciones crónicas y consultas.
Proyecto	Iniciativa social de la organización. Agrupa jornadas y puede recibir donaciones.
Jornada	Brigada médica en una comunidad y fecha determinadas. Es la unidad operativa central.
Atención	Visita de un paciente a una jornada. Registra el recorrido desde la llegada hasta la salida.
Triaje	Toma de signos vitales previa a la consulta.
Consulta	Evaluación médica: motivo, diagnóstico, tratamiento y seguimiento.
Receta	Prescripción emitida en una consulta. Vincula la atención clínica con el inventario.
Principio activo	Componente farmacológico. Permite consultar disponibilidad sin depender de la marca.
Medicamento	Presentación comercial concreta de uno o varios principios activos.
Lote	Cantidad recibida con una fecha de vencimiento propia. Base de la trazabilidad y de las alertas.
Bodega	Ubicación física de almacenamiento, incluidos los botiquines que viajan a cada jornada.
Movimiento	Asiento que registra toda entrada, salida, traslado o ajuste de inventario.
Existencia	Saldo disponible de un lote en una bodega.
Donante y donación	Origen de los recursos e insumos que ingresan a la organización.

Modelo Físico

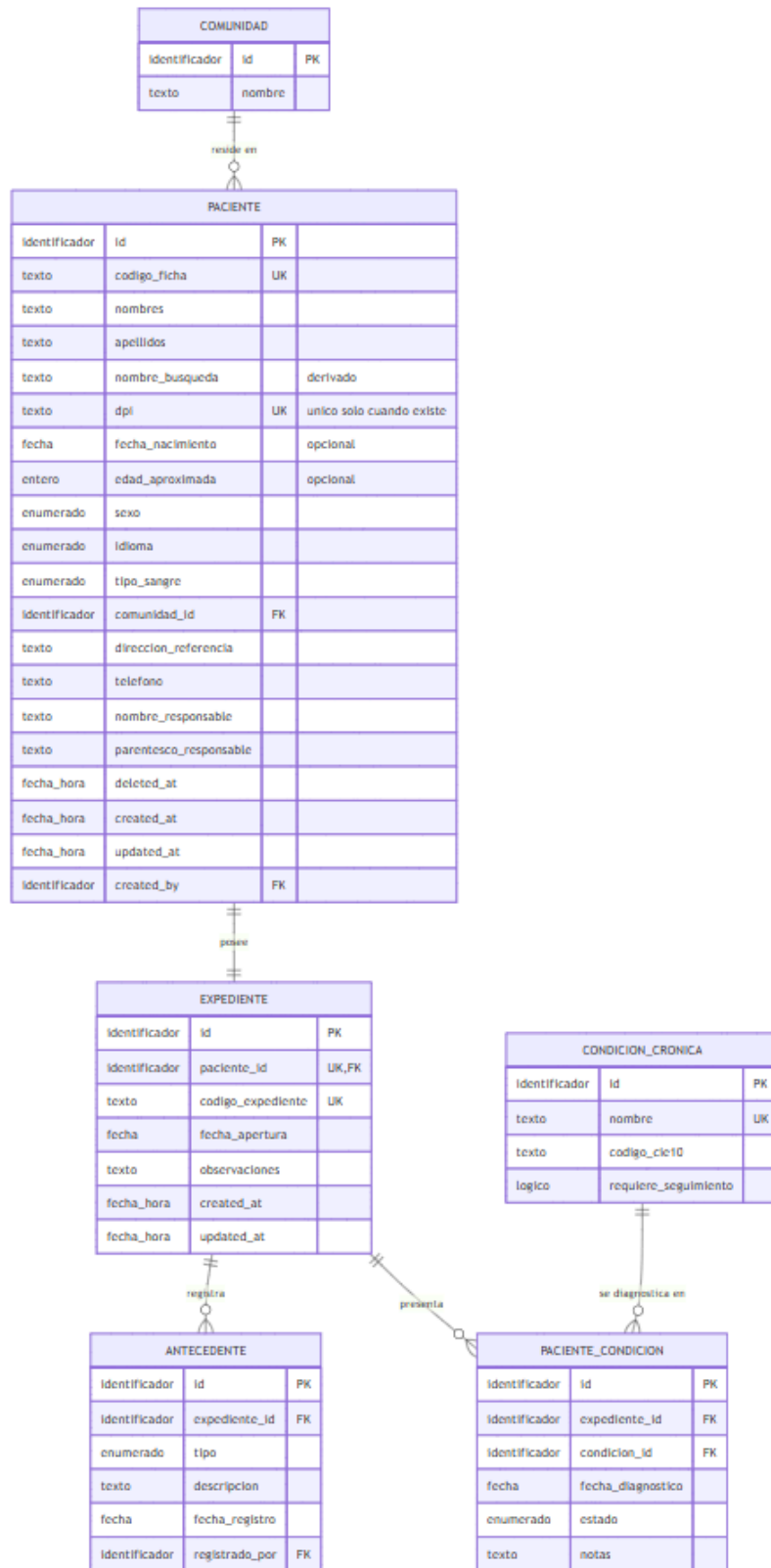
El modelo físico es el más detallado por lo que adjuntamos el link del mismo para tener una mejor visibilidad.

DIAGRAMA LÓGICO

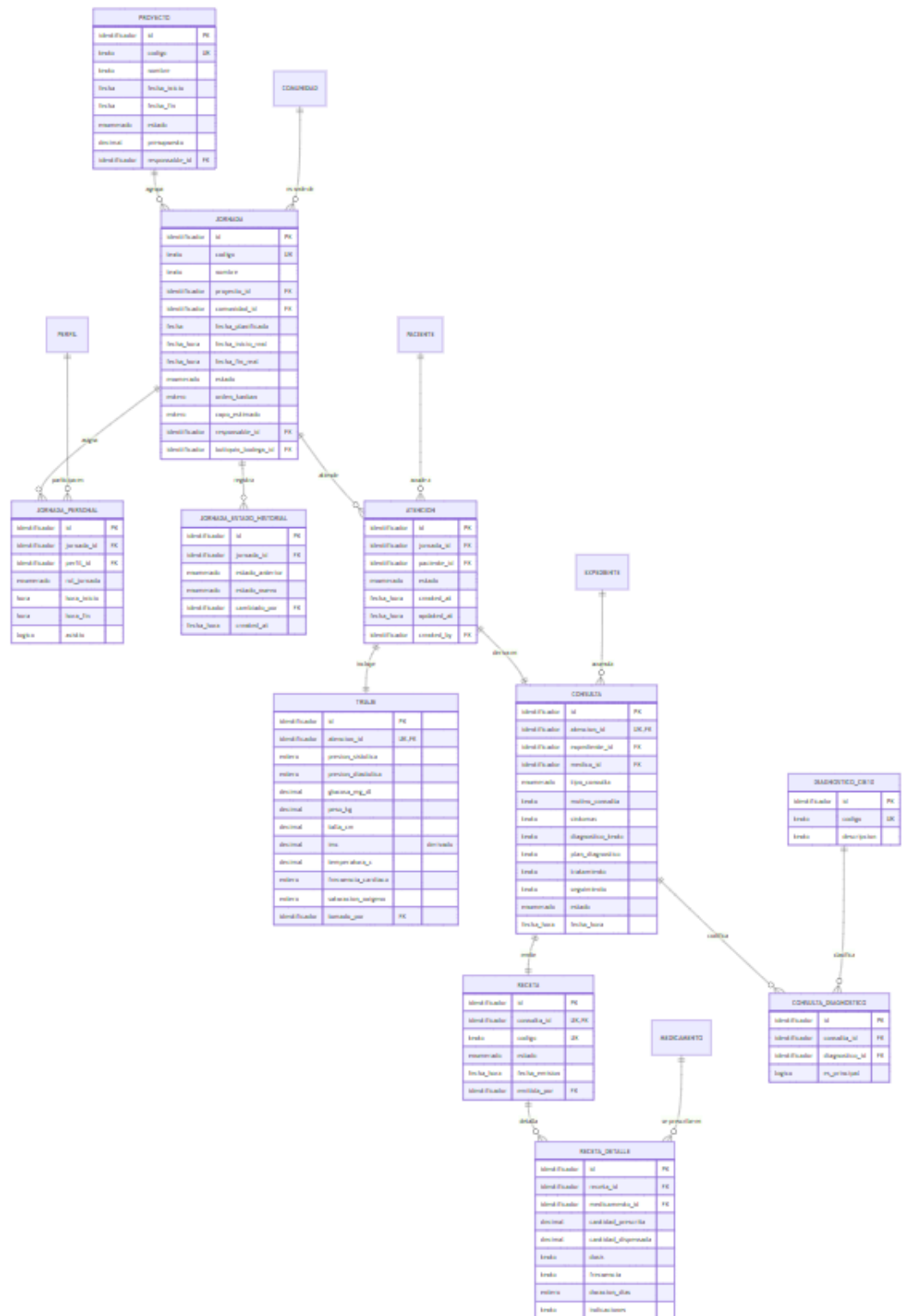
Identidad, acceso, territorio y auditoría



Pacientes y expediente clínico



Jornadas y atención clínica



Inventario, lotes y donaciones

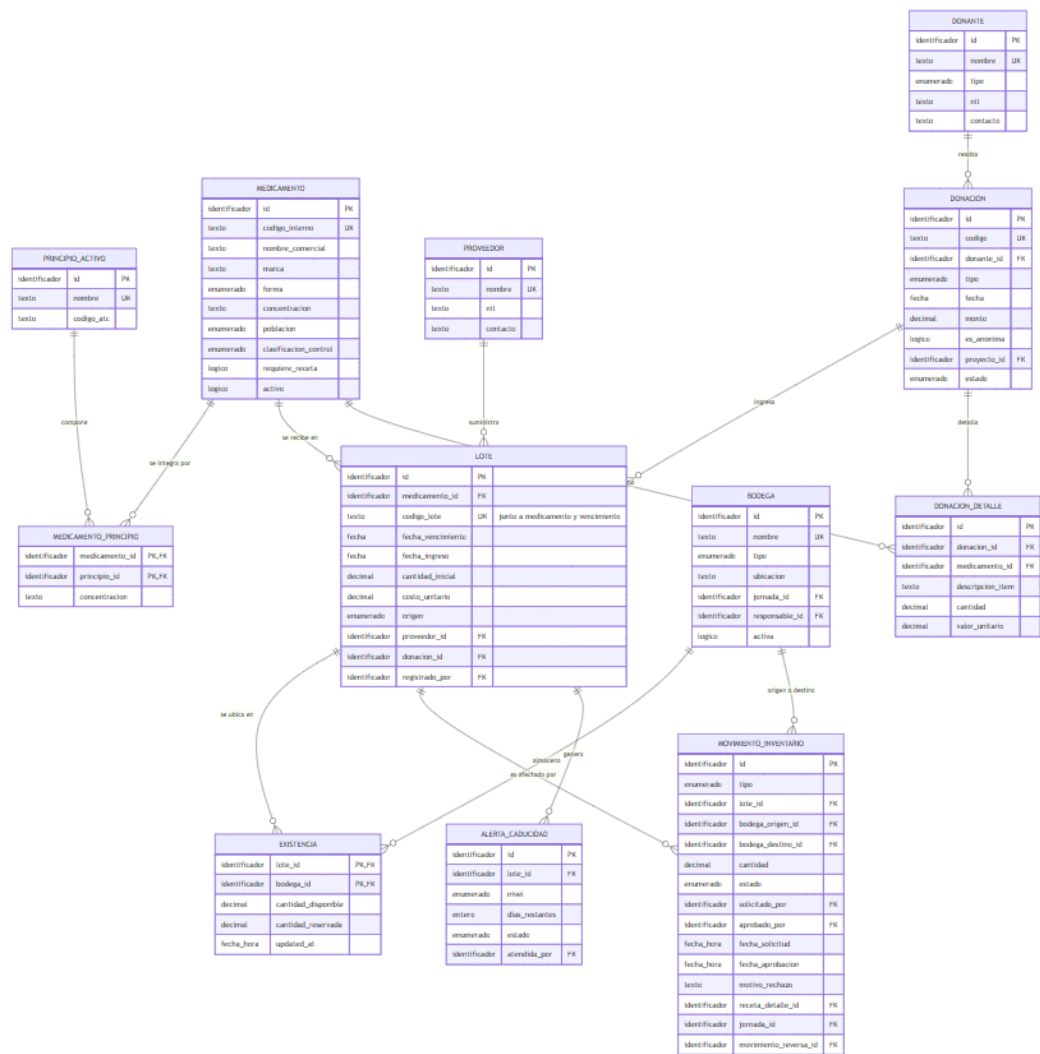
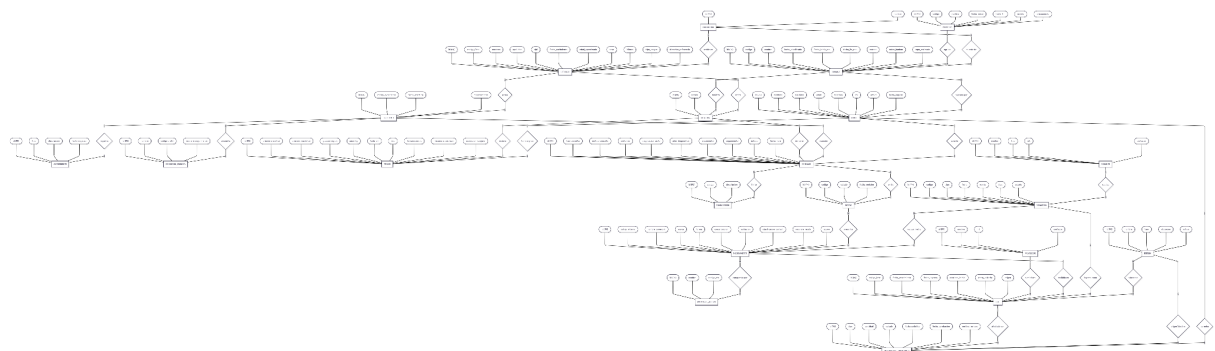


Diagrama Entidad-Relación

DIAGRAMA ENTIDAD-RELACION



Normalización

El modelo de Ecopac Digital se analiza hasta la Tercera Forma Normal (3FN), utilizando tablas intermedias para relaciones muchos-a-muchos y claves foráneas para representar dependencias entre entidades.

Primera Forma Normal (1FN)

El modelo cumple 1FN al mantener atributos atómicos y evitar grupos repetitivos dentro de una misma columna. Las relaciones que podrían contener múltiples valores se separan en entidades asociativas.

- PERFIL_ESPECIALIDAD: permite múltiples especialidades mediante registros separados.
- JORNADA_PERSONAL: separa la participación de cada perfil en una jornada.
- CONSULTA_DIAGNOSTICO: separa los diagnósticos asociados a una consulta.
- MEDICAMENTO_PRINCIPIO: separa los principios activos de un medicamento.
- DONACION_DETALLE: separa los artículos o medicamentos incluidos en una donación.

Segunda Forma Normal (2FN)

Las tablas con claves compuestas se diseñan de modo que sus atributos no clave dependan solamente de una parte de la clave. Esto es especialmente relevante en las entidades asociativas.

- PERFIL_ESPECIALIDAD: la asociación se identifica por (perfil_id, nombre_especialidad).
- ROL_PERMISO: la combinación (rol, permiso_id) identifica la asignación.
- USUARIO_PERMISO: la combinación (perfil_id, permiso_id) identifica el permiso específico.
- MEDICAMENTO_PRINCIPIO: la combinación (medicamento_id, principio_id) identifica la asociación.
- EXISTENCIA: la combinación (lote_id, bodega_id) identifica la existencia de un lote en una bodega.

En estas estructuras, los atributos descriptivos dependen de la asociación completa y no de una parte aislada de la clave.

Tercera Forma Normal (3FN)

El modelo procura que los atributos no clave dependan directamente de la clave primaria y no de otros atributos no clave. Catálogos y entidades independientes se separan para evitar dependencias transitivas.

- DEPARTAMENTO → MUNICIPIO → COMUNIDAD: la jerarquía territorial se separa en entidades relacionadas.
- MEDICAMENTO → PRINCIPIO_ACTIVO: los principios activos se administran como entidad independiente.
- CONSULTA → DIAGNOSTICO_CIE10: los diagnósticos codificados se mantienen en un catálogo independiente.
- DONANTE → DONACION → DONACION_DETALLE: la información del donante, la operación y sus detalles se mantienen separadas.
- LOTE → PROVEEDOR/DONACION: el origen del lote se referencia mediante claves foráneas en lugar de duplicar datos del proveedor o donación.

Consideraciones especiales

- EXISTENCIA representa el estado actual de un lote dentro de una bodega y utiliza una clave compuesta (lote_id, bodega_id).
- MOVIMIENTO_INVENTARIO conserva el historial de operaciones del inventario. La existencia puede considerarse una estructura operativa de consulta rápida, mientras que los movimientos funcionan como trazabilidad.
- En LOTE, el código del ER marca codigo_lote como UK pero incluye la observación de que la unicidad se considera junto con medicamento y vencimiento. Esto debe confirmarse durante la implementación: si la intención es una unicidad compuesta, la restricción debe expresarse explícitamente en el modelo físico.
- En BODEGA y JORNADA existe una relación de botiquín y además dos claves relacionadas con jornada. Se recomienda confirmar con el equipo cuál FK constituye la relación principal antes de generar el script físico definitivo.

Diccionario de Datos

El siguiente diccionario corresponde al Diagrama Entidad-Relacion proporcionado. Contiene las 37 entidades del modelo y conserva los nombres, tipos conceptuales, claves y observaciones identificadas en dicho diagrama.

1. PERFIL

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador único del perfil.
texto	nombres		No	Nombres del usuario.
texto	apellidos		No	Apellidos del usuario.
texto	email	UK	No	Correo electrónico único.
texto	telefono		Sí	Número telefónico.
enumerado	rol		No	Rol asignado al perfil.
logico	activo		No	Indica si el perfil está activo.
fecha	fecha_ingreso		Sí	Fecha de ingreso.
fecha_hora	created_at		No	Fecha y hora de creación.
fecha_hora	updated_at		No	Fecha y hora de última actualización.

2. PERFIL_ESPECIALIDAD

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	perfil_id	PK, FK	No	Perfil al que pertenece la especialidad; referencia PERFIL.id.
texto	nombre_especialidad	PK	No	Especialidad asociada al perfil. La PK es compuesta con perfil_id.

3. PERMISO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador único del permiso.
texto	clave	UK	No	Clave única del permiso.
texto	modulo		No	Módulo funcional al que pertenece.
texto	descripcion		Sí	Descripción del permiso.

4. ROL_PERMISO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
enumerado	rol	PK	No	Rol al que se asigna el permiso.
identificador	permiso_id	PK, FK	No	Permiso asociado; referencia PERMISO.id. PK compuesta.

5. USUARIO_PERMISO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	perfil_id	PK, FK	No	Perfil al que se concede o revoca el permiso.
identificador	permiso_id	PK, FK	No	Permiso asignado al perfil.
logico	concedido		No	Indica si el permiso está concedido.
identificador	otorgado_por	FK	Sí	Perfil que otorgó el permiso; referencia PERFIL.id.
texto	motivo		Sí	Motivo de la asignación o modificación.

6. DEPARTAMENTO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del departamento.
texto	codigo	UK	No	Código único del departamento.
texto	nombre	UK	No	Nombre único del departamento.

7. MUNICIPIO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del municipio.
identificador	departamento_id	FK	No	Departamento al que pertenece; referencia DEPARTAMENTO.id.
texto	nombre		No	Nombre del municipio.

8. COMUNIDAD

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la comunidad.
identificador	municipio_id	FK	No	Municipio al que pertenece; referencia MUNICIPIO.id.
texto	nombre		No	Nombre de la comunidad.

9. AUDITORIA_EVENTO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del evento de auditoría.
texto	tabla_afectada		No	Tabla sobre la que ocurrió la acción.
identificador	registro_id		No	Identificador del registro afectado.

enumerado	accion		No	Tipo de acción realizada.
identificador	perfil_id	FK	Sí	Perfil que generó el evento; referencia PERFIL.id.
estructurado	datos_anteriores		Sí	Estado anterior del registro.
estructurado	datos_nuevos		Sí	Estado posterior del registro.
fecha_hora	created_at		No	Fecha y hora del evento.

10. PACIENTE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador único del paciente.
texto	codigo_ficha	UK	No	Código único de ficha.
texto	nombres		No	Nombres del paciente.
texto	apellidos		No	Apellidos del paciente.
texto	nombre_busqueda		No	Campo derivado para facilitar búsquedas por nombre.
texto	dpi	UK	Sí	Documento de identificación; único cuando existe.
fecha	fecha_nacimiento		Sí	Fecha de nacimiento.
entero	edad_aproximada		Sí	Edad aproximada cuando no se dispone de fecha exacta.
enumerado	sexo		No	Sexo registrado.
enumerado	idioma		Sí	Idioma del paciente.
enumerado	tipo_sangre		Sí	Tipo de sangre.
identificador	comunidad_id	FK	No	Comunidad de residencia; referencia COMUNIDAD.id.

texto	direccion_referencia		Sí	Referencia de ubicación/dirección.
texto	telefono		Sí	Teléfono del paciente.
texto	nombre_responsable		Sí	Nombre del responsable.
texto	parentesco_responsable		Sí	Parentesco del responsable.
fecha_hora	deleted_at		Sí	Fecha de eliminación lógica.
fecha_hora	created_at		No	Fecha y hora de creación.
fecha_hora	updated_at		No	Fecha y hora de actualización.
identificador	created_by	FK	Sí	Perfil que creó el registro; referencia PERFIL.id.

11. EXPEDIENTE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del expediente.
identificador	paciente_id	UK, FK	No	Paciente propietario; referencia PACIENTE.id.
texto	codigo_expediente	UK	No	Código único del expediente.
fecha	fecha_apertura		No	Fecha de apertura.
texto	observaciones		Sí	Observaciones generales.
fecha_hora	created_at		No	Fecha y hora de creación.
fecha_hora	updated_at		No	Fecha y hora de actualización.

12. ANTECEDENTE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del antecedente.
identificador	expediente_id	FK	No	Expediente asociado; referencia EXPEDIENTE.id.
enumerado	tipo		No	Tipo de antecedente.
texto	descripcion		No	Descripción del antecedente.
fecha	fecha_registro		No	Fecha de registro.
identificador	registrado_por	FK	Sí	Perfil que registró el antecedente; referencia PERFIL.id.

13. CONDICION_CRONICA

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la condición.
texto	nombre	UK	No	Nombre único de la condición.
texto	codigo_cie10		Sí	Código CIE-10 asociado.
logico	requiere_seguimiento		No	Indica si requiere seguimiento.

14. PACIENTE_CONDICION

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la asociación.
identificador	expediente_id	FK	No	Expediente asociado; referencia EXPEDIENTE.id.
identificador	condicion_id	FK	No	Condición crónica; referencia CONDICION_CRONICA.id.

fecha	fecha_diagnostico		Sí	Fecha del diagnóstico.
enumerado	estado		No	Estado de la condición.
texto	notas		Sí	Notas relacionadas.

15. PROYECTO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del proyecto.
texto	codigo	UK	No	Código único.
texto	nombre		No	Nombre del proyecto.
fecha	fecha_inicio		Sí	Fecha de inicio.
fecha	fecha_fin		Sí	Fecha de finalización.
enumerado	estado		No	Estado del proyecto.
decimal	presupuesto		Sí	Presupuesto asignado.
identificador	responsable_id	FK	No	Perfil responsable; referencia PERFIL.id.

16. JORNADA

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la jornada.
texto	codigo	UK	No	Código único de la jornada.
texto	nombre		No	Nombre de la jornada.
identificador	proyecto_id	FK	Sí	Proyecto al que pertenece; referencia PROYECTO.id.
identificador	comunidad_id	FK	No	Comunidad sede; referencia COMUNIDAD.id.
fecha	fecha_planificada		Sí	Fecha planificada.
fecha_hora	fecha_inicio_real		Sí	Inicio real.
fecha_hora	fecha_fin_real		Sí	Fin real.
enumerado	estado		No	Estado de la jornada.

entero	orden_kanban		Sí	Orden dentro del tablero Kanban.
entero	cupo_estimado		Sí	Cantidad estimada de atención.
identificador	responsable_id	FK	No	Perfil responsable; referencia PERFIL.id.
identificador	botiquin_bodega_id	FK	Sí	Bodega utilizada como botiquín; referencia BODEGA.id.

17. JORNADA_PERSONAL

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la asignación.
identificador	jornada_id	FK	No	Jornada asociada; referencia JORNADA.id.
identificador	perfil_id	FK	No	Perfil participante; referencia PERFIL.id.
enumerado	rol_jornada		No	Rol desempeñado en la jornada.
hora	hora_inicio		Sí	Hora de inicio de participación.
hora	hora_fin		Sí	Hora de finalización.
logico	asistio		No	Indica si asistió.

18. JORNADA_ESTADO_HISTORIAL

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del cambio.
identificador	jornada_id	FK	No	Jornada modificada; referencia JORNADA.id.
enumerado	estado_anterior		Sí	Estado previo.
enumerado	estado_nuevo		No	Nuevo estado.

identificador	cambiado_por	FK	No	Perfil que realizó el cambio; referencia PERFIL.id.
fecha_hora	created_at		No	Fecha y hora del cambio.

19. ATENCION

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la atención.
identificador	jornada_id	FK	No	Jornada donde ocurre; referencia JORNADA.id.
identificador	paciente_id	FK	No	Paciente atendido; referencia PACIENTE.id.
enumerado	estado		No	Estado de la atención.
fecha_hora	created_at		No	Fecha y hora de creación.
fecha_hora	updated_at		No	Fecha y hora de actualización.
identificador	created_by	FK	Sí	Perfil que creó la atención; referencia PERFIL.id.

20. TRIAJE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del triaje.
identificador	atencion_id	UK, FK	No	Atención asociada; referencia ATENCION.id. Relación 1:1.
entero	presion_sistolica		Sí	Presión sistólica.
entero	presion_diastolica		Sí	Presión diastólica.
decimal	glucosa_mg_dl		Sí	Glucosa en mg/dL.
decimal	peso_kg		Sí	Peso en kilogramos.
decimal	talla_cm		Sí	Talla en centímetros.
decimal	imc		Sí	Índice de masa corporal derivado.

decimal	temperatura_c		Sí	Temperatura en grados Celsius.
entero	frecuencia_cardiaca		Sí	Frecuencia cardíaca.
entero	saturacion_oxigeno		Sí	Saturación de oxígeno.
identificador	tomado_por	FK	Sí	Perfil que tomó los signos; referencia PERFIL.id.

21. CONSULTA

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la consulta.
identificador	atencion_id	UK, FK	No	Atención asociada; referencia ATENCION.id.
identificador	expediente_id	FK	No	Expediente clínico; referencia EXPEDIENTE.id.
identificador	medico_id	FK	No	Perfil médico que atiende; referencia PERFIL.id.
enumerado	tipo_consulta		No	Tipo de consulta.
texto	motivo_consulta		Sí	Motivo de consulta.
texto	sintomas		Sí	Síntomas reportados.
texto	diagnostico_texto		Sí	Descripción textual del diagnóstico.
texto	plan_diagnostico		Sí	Plan diagnóstico.
texto	tratamiento		Sí	Tratamiento indicado.
texto	seguimiento		Sí	Indicaciones de seguimiento.
enumerado	estado		No	Estado de la consulta.
fecha_hora	fecha_hora		No	Fecha y hora de la consulta.

22. DIAGNOSTICO_CIE10

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del diagnóstico.

texto	codigo	UK	No	Código CIE-10 único.
texto	descripcion		No	Descripción del diagnóstico.

23. CONSULTA_DIAGNOSTICO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la asociación.
identificador	consulta_id	FK	No	Consulta; referencia CONSULTA.id.
identificador	diagnostico_id	FK	No	Diagnóstico; referencia DIAGNOSTICO_CIE10.id.
logico	es_principal		No	Indica si es el diagnóstico principal.

24. RECETA

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la receta.
identificador	consulta_id	UK, FK	No	Consulta que emite la receta; referencia CONSULTA.id.
texto	codigo	UK	No	Código único de receta.
enumerado	estado		No	Estado de la receta.
fecha_hora	fecha_emision		No	Fecha y hora de emisión.
identificador	emitida_por	FK	No	Perfil que emitió la receta; referencia PERFIL.id.

25. RECETA_DETALLE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del detalle.
identificador	receta_id	FK	No	Receta; referencia RECETA.id.
identificador	medicamento_id	FK	No	Medicamento prescrito; referencia MEDICAMENTO.id.
decimal	cantidad_prescrita		No	Cantidad prescrita.
decimal	cantidad_dispensada		Sí	Cantidad dispensada.
texto	dosis		No	Dosis indicada.
texto	frecuencia		No	Frecuencia de administración.
entero	duracion_dias		Sí	Duración en días.
texto	indicaciones		Sí	Indicaciones adicionales.

26. PRINCIPIO_ACTIVADO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del principio activo.
texto	nombre	UK	No	Nombre único.
texto	codigo_atc		Sí	Código ATC.

27. MEDICAMENTO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del medicamento.
texto	codigo_interno	UK	No	Código interno único.

texto	nombre_comercial		No	Nombre comercial.
texto	marca		Sí	Marca.
enumerado	forma		No	Forma farmacéutica.
texto	concentracion		Sí	Concentración.
enumerado	poblacion		Sí	Población objetivo.
enumerado	clasificacion_contr ol		Sí	Clasificación de control.
logico	requiere_receta		No	Indica si requiere receta.
logico	activo		No	Indica si está activo.

28. MEDICAMENTO_PRINCIPIO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificado r	medicamento_id	PK, FK	No	Medicamento asociado; referencia MEDICAMENTO.id.
identificado r	principio_id	PK, FK	No	Principio activo asociado; referencia PRINCIPIO_ACTIVADO.id.
texto	concentracion		Sí	Concentración correspondiente a la asociación. PK compuesta.

29. PROVEEDOR

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del proveedor.
texto	nombre	UK	No	Nombre único del proveedor.
texto	nit		Sí	NIT del proveedor.
texto	contacto		Sí	Datos de contacto.

30. BODEGA

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la bodega.
texto	nombre	UK	No	Nombre único.
enumerado	tipo		No	Tipo de bodega.
texto	ubicacion		Sí	Ubicación.
identificador	jornada_id	FK	Sí	Jornada asociada según el diseño; referencia JORNADA.id.
identificador	responsable_id	FK	Sí	Perfil responsable; referencia PERFIL.id.
logico	activa		No	Indica si está activa.

31. LOTE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del lote.
identificador	medicamento_id	FK	No	Medicamento del lote; referencia MEDICAMENTO.id.
texto	codigo_lote	UK	No	Código del lote. El ER anota que la unicidad se considera junto con medicamento y vencimiento; debe confirmarse como restricción compuesta.
fecha	fecha_vencimiento		No	Fecha de vencimiento.
fecha	fecha_ingreso		No	Fecha de ingreso.
decimal	cantidad_inicial		No	Cantidad inicial del lote.
decimal	costo_unitario		Sí	Costo unitario.
enumerado	origen		No	Origen del lote.

identificador	proveedor_id	FK	Sí	Proveedor; referencia PROVEEDOR.id.
identificador	donacion_id	FK	Sí	Donación de origen; referencia DONACION.id.
identificador	registrado_por	FK	Sí	Perfil que registró el lote; referencia PERFIL.id.

32. EXISTENCIA

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	lote_id	PK, FK	No	Lote almacenado; referencia LOTE.id.
identificador	bodega_id	PK, FK	No	Bodega donde se almacena; referencia BODEGA.id.
decimal	cantidad_disponible		No	Cantidad disponible.
decimal	cantidad_reservada		No	Cantidad reservada.
fecha_hora	updated_at		No	Fecha y hora de actualización. PK compuesta lote_id+bodega_id.

33. MOVIMIENTO_INVENTARIO

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del movimiento.
enumerado	tipo		No	Tipo de movimiento.
identificador	lote_id	FK	No	Lote afectado; referencia LOTE.id.
identificador	bodega_origen_id	FK	Sí	Bodega origen; referencia BODEGA.id.

identificado r	bodega_destino_id	FK	Sí	Bodega destino; referencia BODEGA.id.
decimal	cantidad		No	Cantidad movilizada.
enumerado	estado		No	Estado del movimiento.
identificado r	solicitado_por	FK	Sí	Perfil solicitante; referencia PERFIL.id.
identificado r	aprobado_por	FK	Sí	Perfil aprobador; referencia PERFIL.id.
fecha_hora	fecha_solicitud		Sí	Fecha y hora de solicitud.
fecha_hora	fecha_aprobacion		Sí	Fecha y hora de aprobación.
texto	motivo_rechazo		Sí	Motivo si el movimiento fue rechazado.
identificado r	receta_detalle_id	FK	Sí	Detalle de receta que originó salida; referencia RECETA_DETALLE.id.
identificado r	jornada_id	FK	Sí	Jornada donde se consume; referencia JORNADA.id.
identificado r	movimiento_reversa_id	FK	Sí	Movimiento que revierte este movimiento.

34. ALERTA_CADUCIDAD

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la alerta.
identificador	lote_id	FK	No	Lote que genera la alerta; referencia LOTE.id.
enumerado	nivel		No	Nivel de alerta.
entero	dias_restantes		No	Días restantes para vencimiento.
enumerado	estado		No	Estado de la alerta.

identificador	atendida_por	FK	Sí	Perfil que atendió la alerta; referencia PERFIL.id.
---------------	--------------	----	----	--

35. DONANTE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del donante.
texto	nombre	UK	No	Nombre único.
enumerado	tipo		No	Tipo de donante.
texto	nit		Sí	NIT.
texto	contacto		Sí	Datos de contacto.

36. DONACION

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador de la donación.
texto	codigo	UK	No	Código único.
identificador	donante_id	FK	No	Donante; referencia DONANTE.id.
enumerado	tipo		No	Tipo de donación.
fecha	fecha		No	Fecha de la donación.
decimal	monto		Sí	Monto monetario, cuando aplica.
logico	es_anonima		No	Indica si la donación es anónima.
identificador	proyecto_id	FK	Sí	Proyecto asociado; referencia PROYECTO.id.
enumerado	estado		No	Estado de la donación.

37. DONACION_DETALLE

Tipo	Campo	PK/FK/UK	Nulo	Descripción
identificador	id	PK	No	Identificador del detalle.
identificador	donacion_id	FK	No	Donación; referencia DONACION.id.
identificador	medicamento_id	FK	Sí	Medicamento donado; referencia MEDICAMENTO.id.
texto	descripcion_item		Sí	Descripción del artículo donado.
decimal	cantidad		No	Cantidad donada.
decimal	valor_unitario		Sí	Valor unitario.

Scripts Iniciales

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pg_trgm;
CREATE EXTENSION IF NOT EXISTS unaccent;
CREATE EXTENSION IF NOT EXISTS citext;
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TYPE estado_jornada AS ENUM ('planificada', 'en curso', 'finalizada',
'cancelada');
CREATE TYPE estado_movimiento AS ENUM ('pendiente de validacion', 'aprobado',
'rechazado');
CREATE TYPE tipo_movimiento AS ENUM ('ingreso compra', 'ingreso donacion', 'salida
dispensacion', 'salida ajuste', 'traslado');
CREATE TYPE rol_usuario AS ENUM ('administrador', 'junta directiva', 'socio fundador',
'medico', 'voluntario general');
CREATE TYPE presentacion_medicamento AS ENUM ('tableta', 'jarabe', 'capsula',
```

```
'inyectable', 'pomada', 'gotas ophthalmic', 'gotas otic');  
CREATE TYPE idioma_preferido AS ENUM ('espanol', 'quiche', 'mam', 'otros');
```

```
CREATE OR REPLACE FUNCTION actualizar_timestamp_updated_at()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.updated_at = NOW();  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TABLE perfiles (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    auth_id UUID UNIQUE NOT NULL REFERENCES auth.users(id),  
    email CITEXT UNIQUE NOT NULL,  
    nombres VARCHAR(100) NOT NULL,  
    apellidos VARCHAR(100) NOT NULL,  
    telefono VARCHAR(20),  
    especialidad VARCHAR(100),  
    rol ROL_USUARIO NOT NULL DEFAULT 'voluntario general',  
    activo BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ  
);
```

```
CREATE TABLE departamentos (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    codigo VARCHAR(10) UNIQUE NOT NULL,  
    nombre VARCHAR(100) NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE municipios (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  departamento_id UUID NOT NULL REFERENCES departamentos(id) ON DELETE  
  RESTRICT,  
  codigo VARCHAR(10) UNIQUE NOT NULL,  
  nombre VARCHAR(100) NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE comunidades (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  municipio_id UUID NOT NULL REFERENCES municipios(id) ON DELETE  
  RESTRICT,  
  nombre VARCHAR(100) NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE pacientes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  perfil_creador_id UUID NOT NULL REFERENCES perfiles(id),  
  comunidad_residencia_id UUID NOT NULL REFERENCES comunidades(id) ON  
  DELETE RESTRICT,  
  nombre_completo VARCHAR(200) NOT NULL,  
  dpi VARCHAR(20) UNIQUE,  
  fecha_nacimiento DATE,  
  edad_aproximada_anios INTEGER,  
  edad_aproximada_meses INTEGER,  
  es_menor_edad BOOLEAN GENERATED ALWAYS AS (  
    CASE  
      WHEN fecha_nacimiento IS NOT NULL THEN (EXTRACT(YEAR FROM
```

```

AGE(fecha_nacimiento)) < 18)
    WHEN edad_aproximada_anios IS NOT NULL THEN (edad_aproximada_anios <
18)
    ELSE NULL
END
) STORED,
genero VARCHAR(10),
idioma IDIOMA_PREFERIDO,
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ,
deleted_at TIMESTAMPTZ,
CONSTRAINT chk_nacimiento_o_edad CHECK (fecha_nacimiento IS NOT NULL OR
(edad_aproximada_anios IS NOT NULL OR edad_aproximada_meses IS NOT NULL))
);

```

```

CREATE TABLE expedientes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    paciente_id UUID UNIQUE NOT NULL REFERENCES pacientes(id) ON DELETE
RESTRICT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE condiciones_cronicas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    nombre VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE padecimientos_cronicos (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    expediente_id UUID NOT NULL REFERENCES expedientes(id),

```

```
condicion_id UUID NOT NULL REFERENCES condiciones_cronicas(id),
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
UNIQUE(expediente_id, condicion_id)
);
```

```
CREATE TABLE proyectos (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre VARCHAR(100) NOT NULL,
  descripcion TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ
);
```

```
CREATE TABLE jornadas (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  proyecto_id UUID NOT NULL REFERENCES proyectos(id),
  responsable_id UUID NOT NULL REFERENCES perfiles(id),
  comunidad_ubicacion_id UUID NOT NULL REFERENCES comunidades(id) ON
DELETE RESTRICT,
  nombre VARCHAR(100) NOT NULL,
  fecha_inicio DATE NOT NULL,
  fecha_fin DATE NOT NULL,
  estado ESTADO_JORNADA NOT NULL DEFAULT 'planificada',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ,
  CONSTRAINT chk_fechas_jornada CHECK (fecha_fin >= fecha_inicio)
);
```

```
CREATE TABLE atenciones (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  jornada_id UUID NOT NULL REFERENCES jornadas(id),
```

```
paciente_id UUID NOT NULL REFERENCES pacientes(id) ON DELETE RESTRICT,  
voluntario_recepcion_id UUID NOT NULL REFERENCES perfiles(id),  
fecha_atencion TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
CONSTRAINT uq_paciente_jornada UNIQUE (jornada_id, paciente_id)  
);
```

```
CREATE TABLE triajes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  atencion_id UUID NOT NULL REFERENCES atenciones(id),  
  presion_sistolica INTEGER,  
  presion_diastolica INTEGER,  
  glucosa_mg_dl INTEGER,  
  peso_kg DECIMAL(5,2),  
  talla_cm INTEGER,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  CONSTRAINT chk_presion CHECK (presion_sistolica > presion_diastolica)  
);
```

```
CREATE TABLE consultas (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  atencion_id UUID NOT NULL REFERENCES atenciones(id),  
  medico_id UUID NOT NULL REFERENCES perfiles(id),  
  motivo TEXT NOT NULL,  
  diagnostico_principal TEXT NOT NULL,  
  tratamiento TEXT NOT NULL,  
  observaciones TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE principios_activos (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
nombre VARCHAR(100) UNIQUE NOT NULL,  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE medicamentos (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  nombre_comercial VARCHAR(100) NOT NULL,  
  presentacion PRESENTACION_MEDICAMENTO NOT NULL,  
  concentración VARCHAR(50),  
  es_controlado BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE medicamentos_principios (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  medicamento_id UUID NOT NULL REFERENCES medicamentos(id),  
  principio_activo_id UUID NOT NULL REFERENCES principios_activos(id),  
  UNIQUE(medicamento_id, principio_activo_id)  
);
```

```
CREATE TABLE bodegas (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  jornada_asociada_id UUID REFERENCES jornadas(id) ON DELETE RESTRICT,  
  nombre VARCHAR(100) NOT NULL,  
  es_fija BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  CONSTRAINT chk_bodega_jornada CHECK ((es_fija = TRUE AND  
jornada_asociada_id IS NULL) OR (es_fija = FALSE AND jornada_asociada_id IS NOT  
NULL))  
);
```

```
CREATE TABLE lotes (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  medicamento_id UUID NOT NULL REFERENCES medicamentos(id) ON DELETE  
RESTRICT,  
  codigo VARCHAR(50) NOT NULL,  
  fecha_vencimiento DATE NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE(medicamento_id, codigo, fecha_vencimiento)  
);
```

```
CREATE TABLE movimientos_inventario (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  perfil_creador_id UUID NOT NULL REFERENCES perfiles(id),  
  jornada_origen_id UUID REFERENCES jornadas(id),  
  bodega_origen_id UUID REFERENCES bodegas(id),  
  bodega_destino_id UUID REFERENCES bodegas(id),  
  lote_id UUID NOT NULL REFERENCES lotes(id) ON DELETE RESTRICT,  
  perfil_validador_id UUID REFERENCES perfiles(id),  
  tipo TIPO_MOVIMIENTO NOT NULL,  
  cantidad INTEGER NOT NULL,  
  estado ESTADO_MOVIMIENTO NOT NULL DEFAULT 'pendiente de validacion',  
  fecha_movimiento TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  fecha_validacion TIMESTAMPTZ,  
  motivo_rechazo TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  CONSTRAINT chk_cantidad_positiva CHECK (cantidad > 0),  
  CONSTRAINT chk_origen_destino CHECK (bodega_origen_id IS DISTINCT FROM  
bodega_destino_id),  
  CONSTRAINT chk_aprobacion_auditable CHECK ((estado = 'aprobado' AND  
perfil_validador_id IS NOT NULL AND fecha_validacion IS NOT NULL) OR (estado <>  
'aprobado'))),
```

```
        CONSTRAINT chk_rechazo_con_motivo CHECK ((estado = 'rechazado' AND
motivo_rechazo IS NOT NULL) OR (estado <> 'rechazado'))
);
```

```
CREATE TABLE existencias (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    bodega_id UUID NOT NULL REFERENCES bodegas(id) ON DELETE RESTRICT,
    lote_id UUID NOT NULL REFERENCES lotes(id) ON DELETE RESTRICT,
    cantidad INTEGER NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ,
    UNIQUE(bodega_id, lote_id),
    CONSTRAINT chk_existencia_no_negativa CHECK (cantidad >= 0)
);
```

```
CREATE TABLE recetas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    consulta_id UUID UNIQUE NOT NULL REFERENCES consultas(id),
    movimiento_dispensacion_id UUID NOT NULL REFERENCES
movimientos_inventario(id) ON DELETE RESTRICT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE OR REPLACE FUNCTION fn_audit_pacientes_soft_delete()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.deleted_at IS NOT NULL AND OLD.deleted_at IS NULL THEN
        PERFORM NULL
        FROM consultas c
        JOIN atenciones a ON c.atencion_id = a.id
        WHERE a.paciente_id = OLD.id;
```

```

        IF FOUND THEN
            RAISE EXCEPTION 'No se puede eliminar un paciente con consultas medicas
asociadas. Use ON DELETE RESTRICT en FKeys clínicas.';
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_audit_pacientes_soft_delete
BEFORE UPDATE ON pacientes
FOR EACH ROW
EXECUTE FUNCTION fn_audit_pacientes_soft_delete();

CREATE OR REPLACE FUNCTION fn_actualizar_existencias()
RETURNS TRIGGER AS $$
DECLARE
    existencia_id UUID;
BEGIN
    IF NEW.estado = 'aprobado' AND OLD.estado <> 'aprobado' THEN
        IF NEW.tipo IN ('ingreso compra', 'ingreso donacion') THEN
            IF NEW.bodega_destino_id IS NULL THEN
                RAISE EXCEPTION 'Ingreso aprobado exige bodega de destino.';
            END IF;
            INSERT INTO existencias (bodega_id, lote_id, cantidad, created_at, updated_at)
                VALUES (NEW.bodega_destino_id, NEW.lote_id, NEW.cantidad, NOW(),
NOW())
            ON CONFLICT (bodega_id, lote_id)
                DO UPDATE SET cantidad = existencias.cantidad + NEW.cantidad, updated_at =
NOW();
            ELSIF NEW.tipo IN ('salida dispensacion', 'salida ajuste') THEN

```

```
IF NEW.bodega_origen_id IS NULL THEN
    RAISE EXCEPTION 'Salida aprobada exige bodega de origen.';
END IF;

UPDATE existencias

SET cantidad = existencias.cantidad - NEW.cantidad, updated_at = NOW()
WHERE bodega_id = NEW.bodega_origen_id AND lote_id = NEW.lote_id;

IF NOT FOUND THEN
    RAISE EXCEPTION 'No se encontraron existencias en bodega de origen para la
salida.';
END IF;

ELSIF NEW.tipo = 'traslado' THEN
    IF NEW.bodega_origen_id IS NULL OR NEW.bodega_destino_id IS NULL
THEN
        RAISE EXCEPTION 'Traslado aprobado exige bodega de origen y destino.';
    END IF;

    UPDATE existencias

    SET cantidad = existencias.cantidad - NEW.cantidad, updated_at = NOW()
    WHERE bodega_id = NEW.bodega_origen_id AND lote_id = NEW.lote_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'No se encontraron existencias en bodega de origen para el
traslado.';
    END IF;

    INSERT INTO existencias (bodega_id, lote_id, cantidad, created_at, updated_at)
        VALUES (NEW.bodega_destino_id, NEW.lote_id, NEW.cantidad, NOW(),
NOW())
    ON CONFLICT (bodega_id, lote_id)
        DO UPDATE SET cantidad = existencias.cantidad + NEW.cantidad, updated_at =
NOW();

    END IF;
END IF;

RETURN NEW;
```

```
END;

$$ LANGUAGE plpgsql;

CREATE TRIGGER tr_actualizar_existencias
AFTER UPDATE ON movimientos_inventario
FOR EACH ROW
EXECUTE FUNCTION fn_actualizar_existencias();

CREATE TRIGGER tr_perfiles_timestamp
BEFORE UPDATE ON perfiles
FOR EACH ROW
EXECUTE FUNCTION actualizar_timestamp_updated_at();

CREATE TRIGGER tr_pacientes_timestamp
BEFORE UPDATE ON pacientes
FOR EACH ROW
EXECUTE FUNCTION actualizar_timestamp_updated_at();

CREATE TRIGGER tr_proyectos_timestamp
BEFORE UPDATE ON proyectos
FOR EACH ROW
EXECUTE FUNCTION actualizar_timestamp_updated_at();

CREATE TRIGGER tr_jornadas_timestamp
BEFORE UPDATE ON jornadas
FOR EACH ROW
EXECUTE FUNCTION actualizar_timestamp_updated_at();

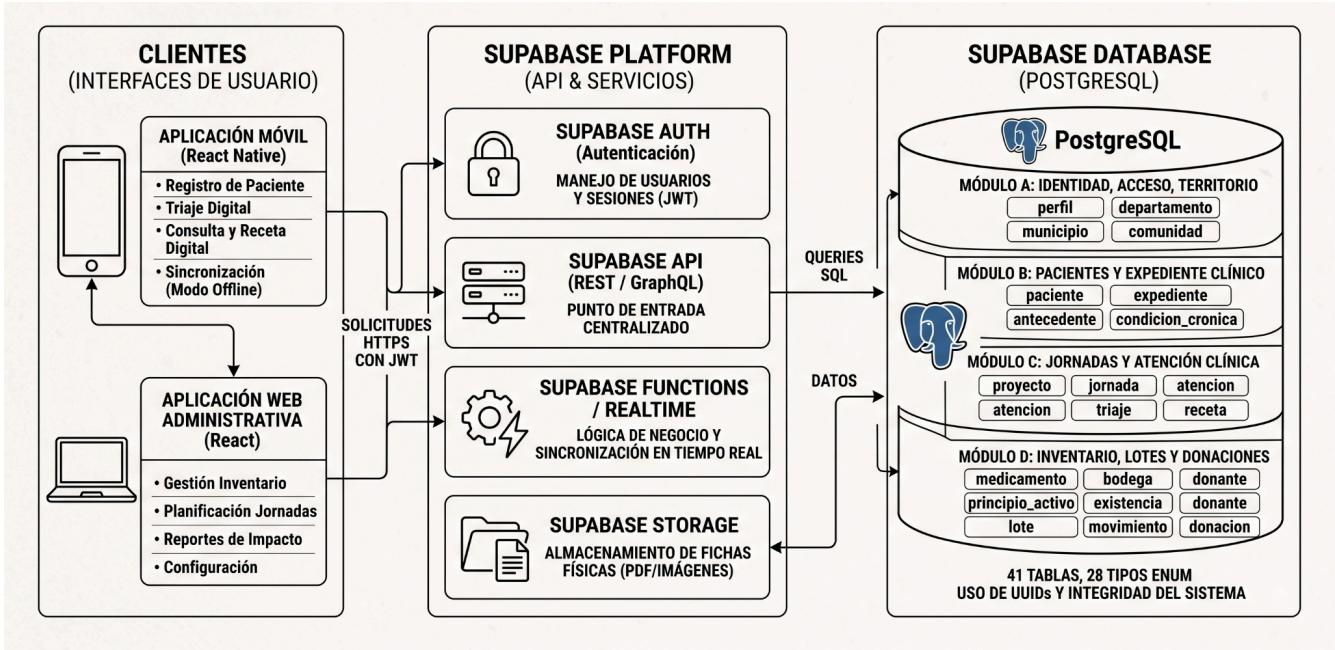
CREATE TRIGGER tr_existencias_timestamp
BEFORE UPDATE ON existencias
FOR EACH ROW
```

```
EXECUTE FUNCTION actualizar_timestamp_updated_at();
```

Diseño de API, Rutas, Servicios e Integraciones

Entidad / Recurso (Tabla)	Ruta Base (Supabase URL)	Métodos (CRUD)	Lógica de Negocio Principal
Autenticación	/auth/v1/*	POST (Login/Logout)	Gestionado por Supabase Auth (JWT).
Pacientes	/rest/v1/paciente	GET, POST, PATCH	Buscar paciente por similitud de nombre, registrar nuevo ingreso.
Expedientes	/rest/v1/expediente	GET, POST, PATCH	Visualizar el historial clínico (solo Médicos/Admin).
Jornadas	/rest/v1/jornada	GET, POST, PATCH	Kanban (cambio de estados), asignación de voluntarios.
Consultas	/rest/v1/consulta	GET, POST	Registrar diagnóstico y receta. Asociar a un paciente y a una jornada.
Inventario (Existencias)	/rest/v1/existencia	GET	Consulta de stock en tiempo real y botiquines activos.
Movimientos (Inventario)	/rest/v1/movimiento_inventario	GET, POST, PATCH	Médicos registran salidas (estado 'pendiente'). Admin los aprueba (PATCH).

Reportes SQL)	(Vistas /rest/v1/vista_reporte_impacto	GET	Acceso de solo lectura a vistas SQL pre-calculadas en PostgreSQL.
---------------	--	-----	---



Diseño de Roles y Permisos

Estructura general:

- Mínimo Privilegio: Cada rol tiene acceso exclusivo a las funciones y datos necesarios para su labor.
- Seguridad a Nivel de Fila (RLS): Las tablas críticas como consulta, perfil e inventario utilizan políticas RLS para restringir el acceso basado en la identidad y rol del usuario (auth.uid() y rol del perfil).
- Jerarquía de Roles: Existe una estructura jerárquica donde los roles administrativos heredan capacidades de lectura, pero se limitan en acciones críticas sobre datos clínicos.

Estructura de Roles

Rol	Descripción y Ámbito de Acción
Administrador (Presidenta)	Acceso total y global a todo el sistema. Gestión de usuarios, inventario, bodegas, proyectos y reportes de impacto. Única autorizada para aprobar movimientos de inventario y datos sensibles.
Junta Directiva	Rol consultivo. Acceso de lectura a reportes de impacto, dashboard, proyectos y listado de pacientes. No tiene permisos de creación o modificación.
Médico	Ámbito operativo en jornadas. Creación de pacientes, registro de triaje, consultas médicas y prescripciones. Lectura de inventario y botiquines activos. No accede a reportes administrativos.
Voluntario	Rol de apoyo en campo. Búsqueda y creación de pacientes, registro de signos vitales. Visualización limitada del stock de la jornada. No accede a diagnósticos ni consultas médicas completas.

Matriz de Permisos por Rol

C = Crear

R = Leer

U = Actualizar

D = Borrar (eliminación técnica)

Módulo / Tabla	Administrador	Junta Directiva / Socio	Médico	Voluntario de Salud / Staff	Políticas RLS Clave
Usuarios (perfil)	CRUD	R	R (solo propio)	R (solo propio)	Administrador puede ver y editar todos. Otros solo el propio.
Territorio (comunidad, etc.)	CRUD	R	R	R	Lectura pública para autenticados. Solo Admin edita.
Pacientes (paciente)	CRUD	R	CRU	CR	Médico/Staff crean. Admin edita. RLS para lectura en jornada.
Expediente (expediente)	CRU	R	CRU	-	Solo Médico y Admin acceden. Staff no ve expediente.
Triaje (triaje)	CR	-	CRU	CRU	Médico y Staff registran.

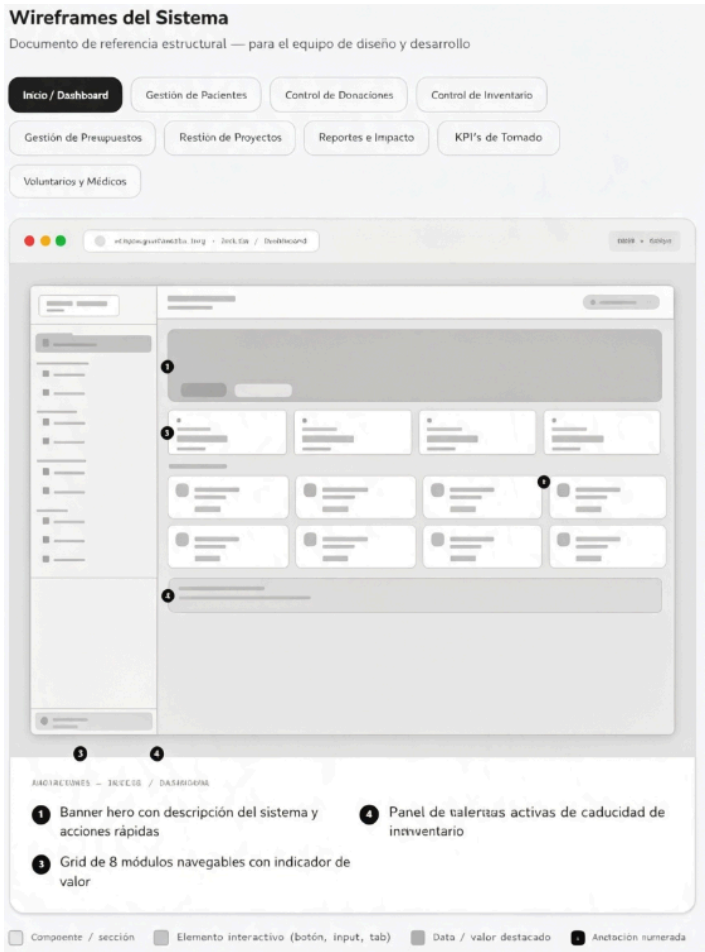
					Admin ve para control.
Consultas (consulta)	CR	-	CRU	-	Solo el Médico que atiende puede editar. Admin ve para auditoría.
Recetas (receta)	CR	-	CR	-	El Médico prescribe. Admin ve para validación.
Jornadas (jornada)	CRUD	R	R (activas)	R (activas)	Solo Admin gestiona. RLS para ver solo jornadas asignadas/activas.
Inventario (medicamento, lote, existencia)	CRUD	R (stock total)	R (stock total)	R (botiquín jornada)	RLS restringe al Staff ver solo botiquines de su jornada.
Movimientos (movimiento_inventario)	CRU (Aprobar)	-	CR (Crear Salida)	CR (Crear Salida)	Médico/Staff crean movimientos

					en estado 'Pendiente'. Solo Admin actualiza a 'Aprobado'.
Donaciones / Proyectos (donacion, etc.)	CRUD	R	-	-	Acceso exclusivo para roles administrativos.

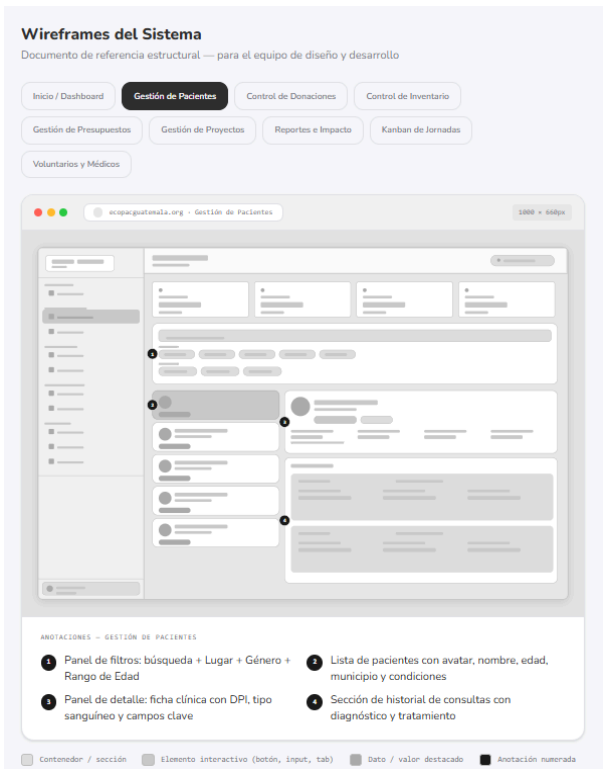
Wireframes

Web

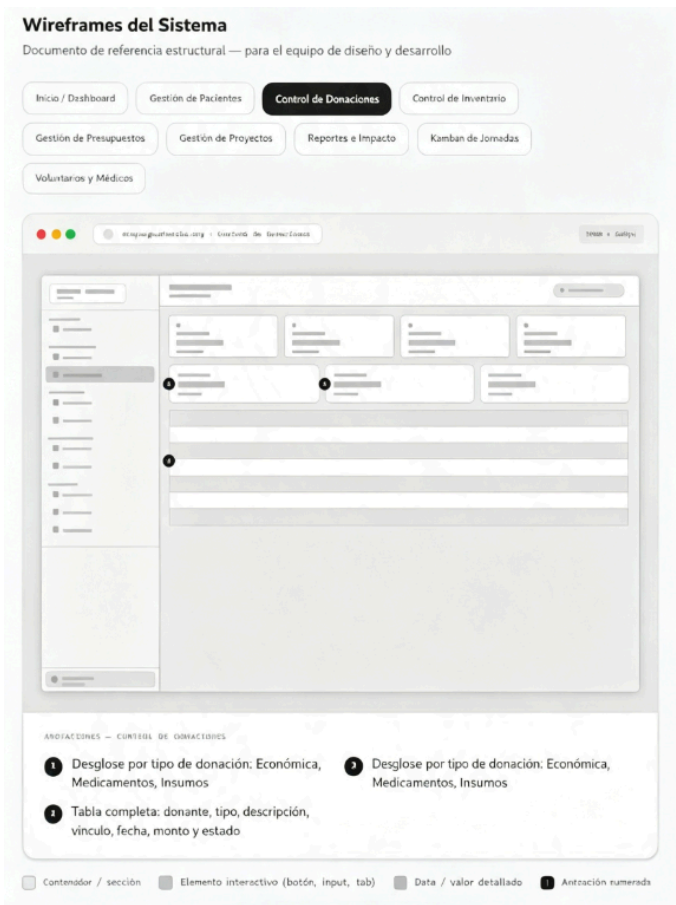
Wireframe inicio/dashboard



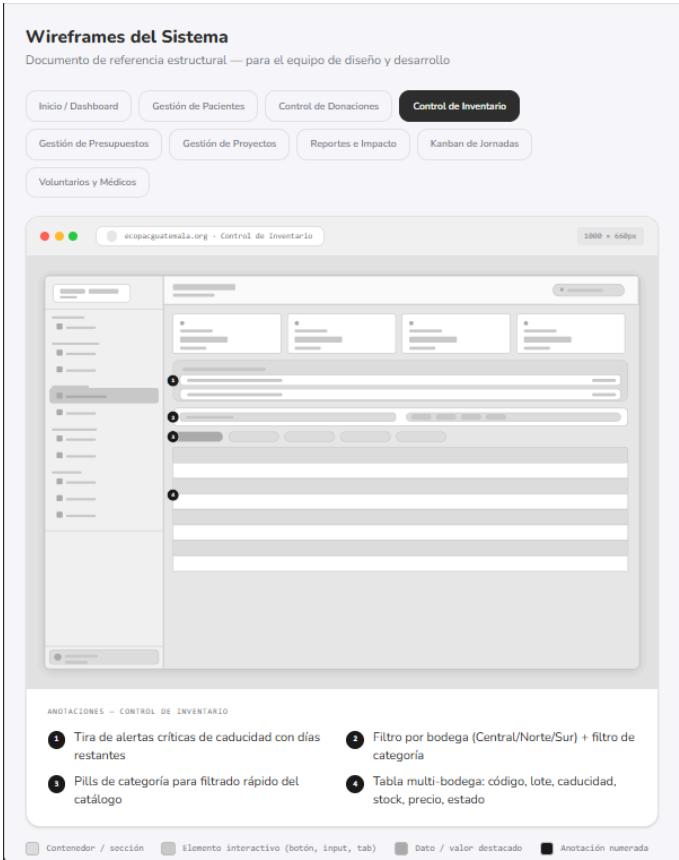
Wireframe gestión de pacientes



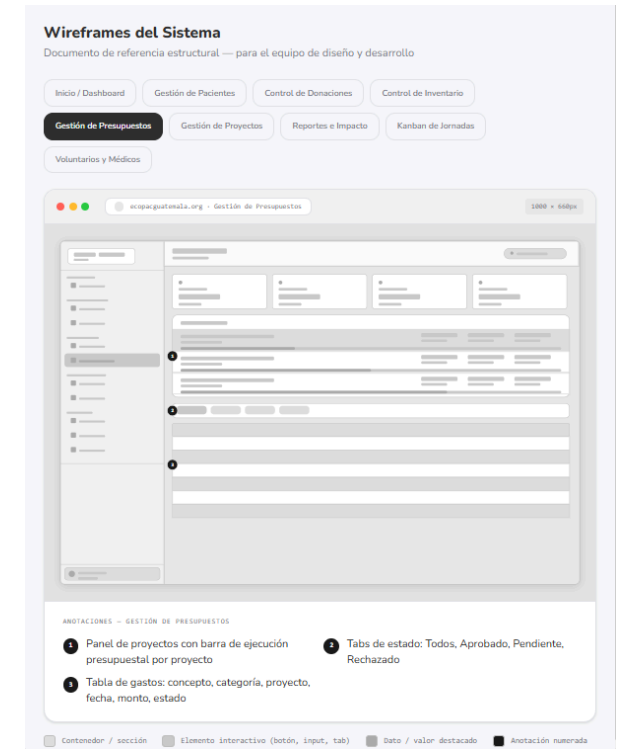
Wireframe control de donaciones



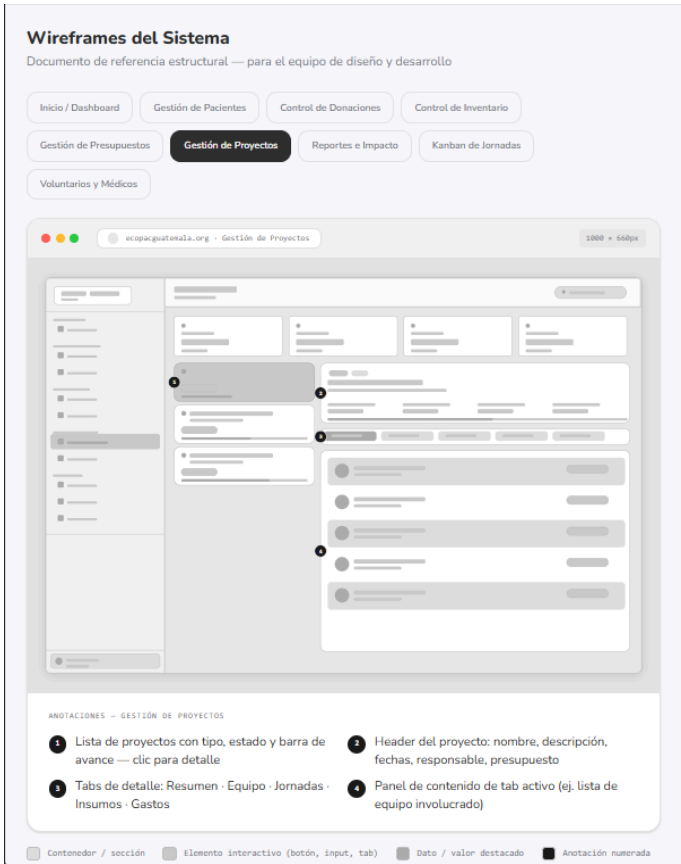
Wireframe control de inventario



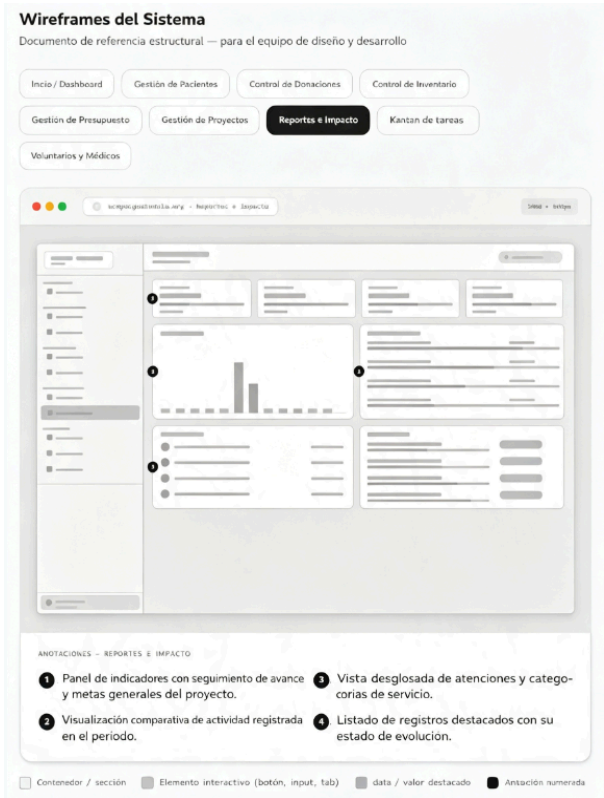
Wireframe gestión de presupuestos



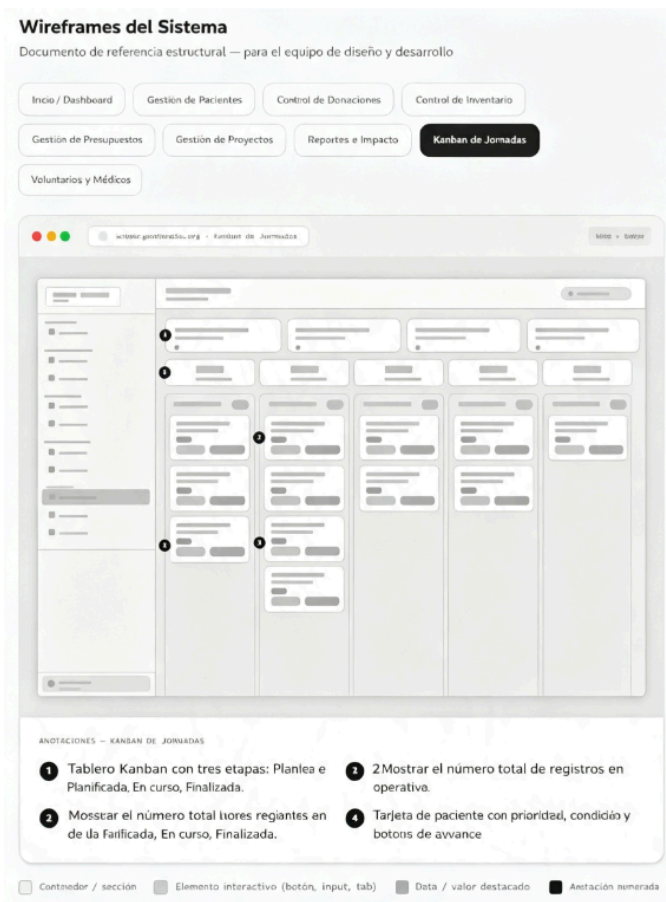
Wireframe gestión de proyectos



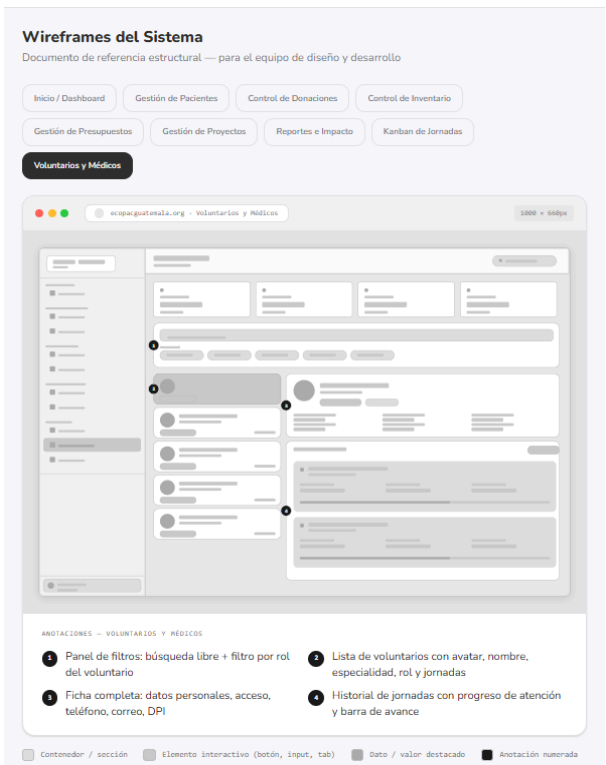
Wireframe reportes de impacto



Wireframe kanban de jornadas

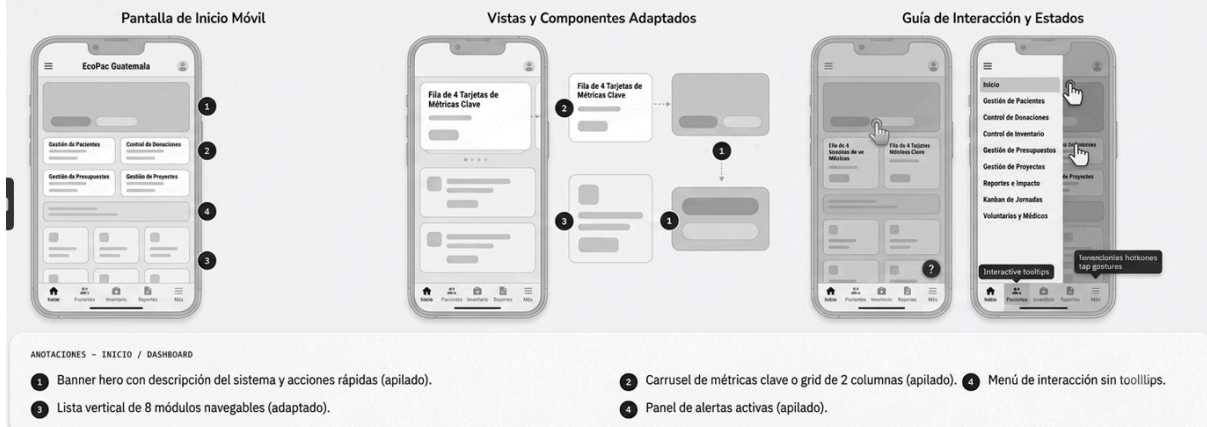


Wireframe voluntarios y médicos



Móvil

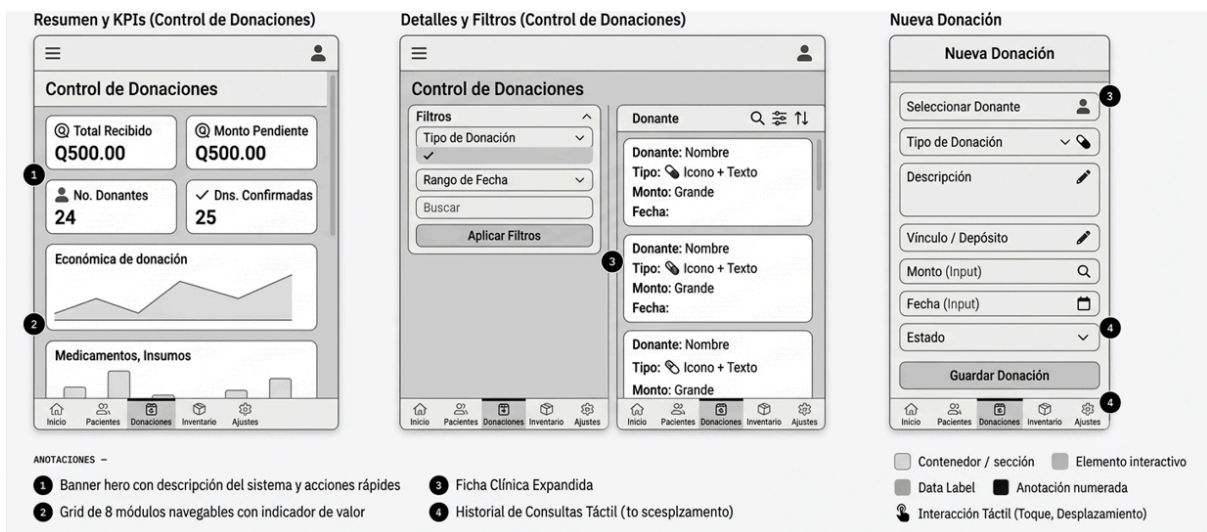
Documento de Referencia Estructural — para Dispositivos Móviles



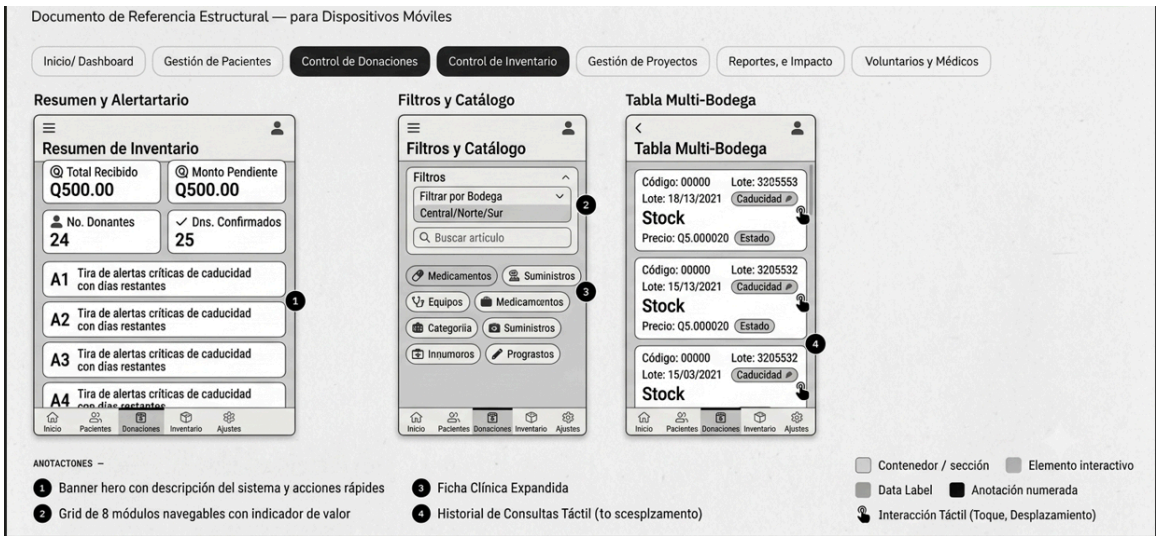
Wireframe gestión de pacientes



Wireframe control de donaciones



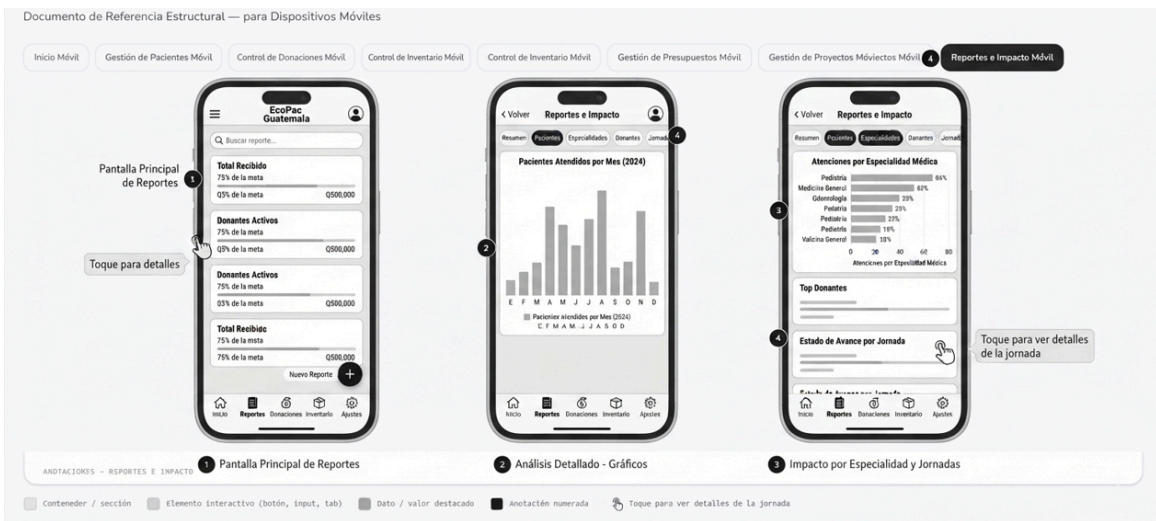
Wireframe control de inventario



Wireframe gestión de proyectos



Wireframe reportes de impacto



Documento de Referencia Estructural — para Dispositivos Móviles

Inicio / Dashboard Gestión de Pacientes Control de Donaciones Control de Inventario Gestión de Presupuestos Gestión de Proyectos Reportes e Impacto Kanban de Impadas 3 Kanban de Jornadas

1. Contenedor / sección

2. Elemento interactivo (botón, input, tab)

3. Date / valor destacado

4. Anotación numerada

ANOTACIONES - KANBAN de Principales de Jornadas

1. Sistema activo

2. Tablero Kanban - Vista de Etapas

3. Panel de Detalle del Paciente

Documento de Referencia Estructural — para Dispositivos Móviles

Inicio / Dashboard Gestión de Pacientes Control de Donaciones Control de Inventario Gestión de Presupuestos Gestión de Proyectos Reportes e Impacto Kanban de Jornadas **Voluntarios y Médicos**

Pantalla Principal de Voluntarios

1. Buscar voluntario...
2. Rol: Médico, Voluntario
3. Dr. Juan Pérez, Médico Gen. Especialidad
4. Ana Gómez, Vol. Logística
5. Ana Gómez, Vol. Logística
6. Dr. Juan Pérez

7. Toque para ver ficha

Ficha Completa del Voluntario

1. Volver
2. Ficha Voluntario
3. Dr. Juan Pérez, Médico Gen. Especialidad
4. Datos Personales, Teléfono, Correo, DPI
5. Acceso al Sistema, Estado, Permisos

Historial de Jornadas

1. Volver
2. Historial Activo
3. Datos, Historial, Jornadas
4. Jornada Name, 100%, 45 pacientes
5. Jornada Name, 100%, 45 pacientes
6. Jornada Name, 50%, 45 pacientes
7. Jornada Name

8. Toque para ver detalles de la jornada

ANTICAPITULO - VOLUNTARIOS y MÉDICOS

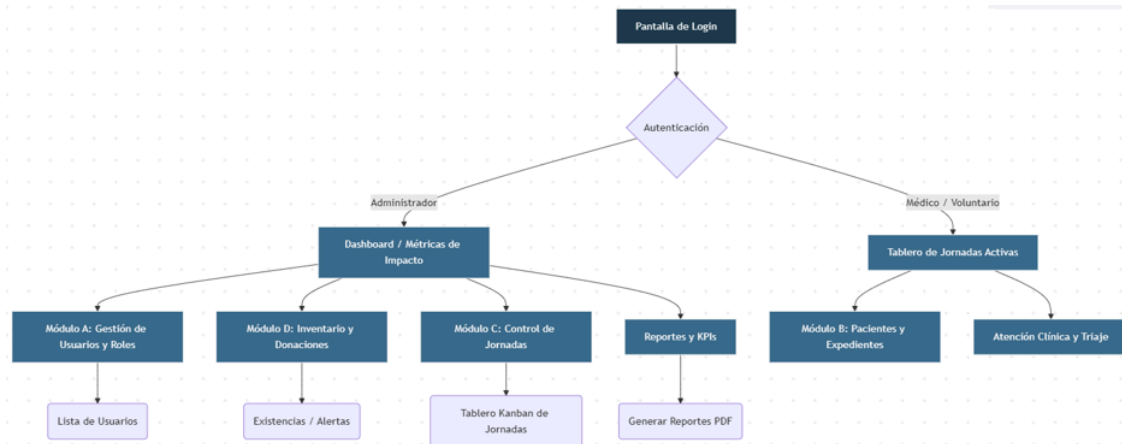
1. Panel de filtros: búsqueda libre + filtro so, teléfono, correo, DPI
2. Ficha Completa del Voluntario
3. Historial de Jornadas

Contenedor / sección Elemento interactivo (botón, input, tab) Datos / valor destacado Añetación numerada ¿Este ra ver details

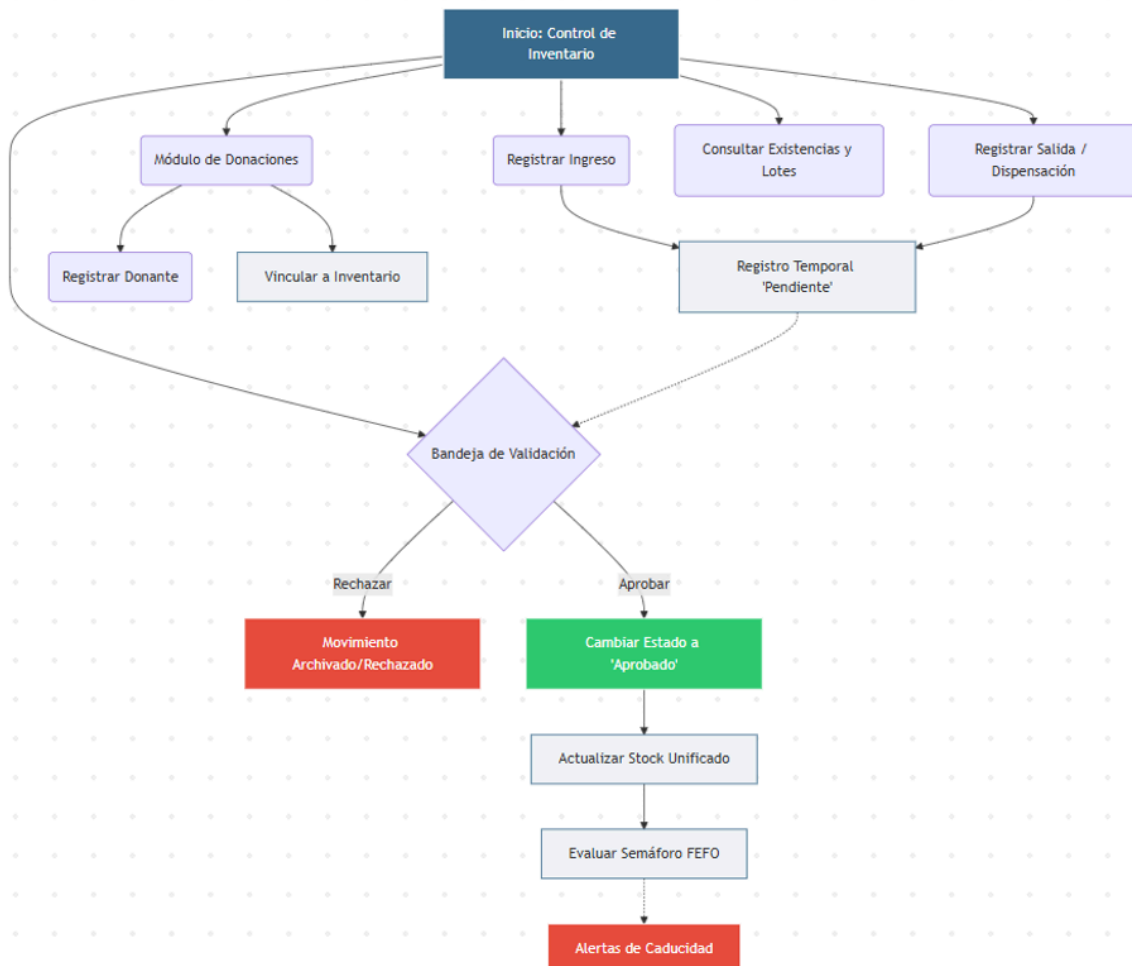
PROTOTIPO

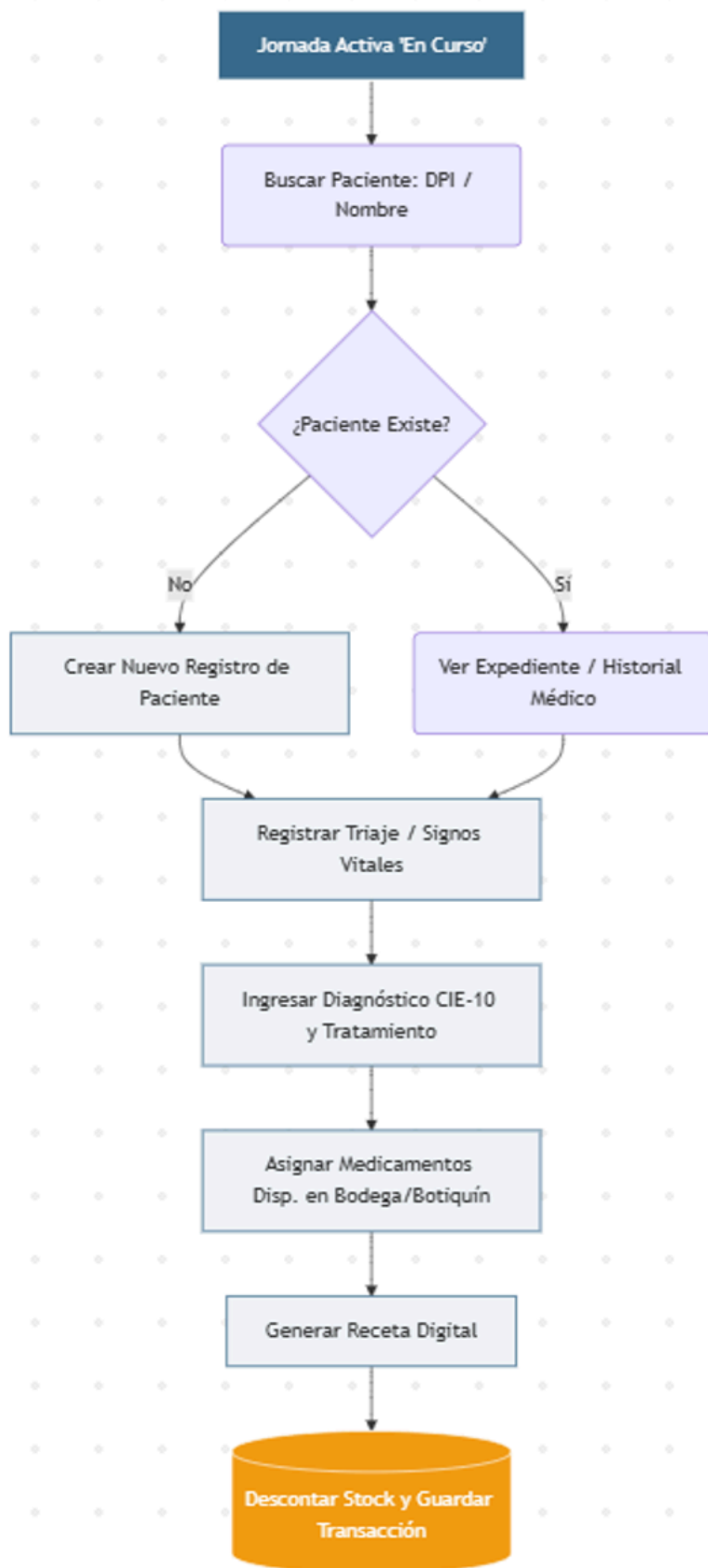
Mapa de Navegación

Mapa General



Mapas Internos





Consideraciones de Seguridad y Protección de Datos

Marco Legal Aplicable en Guatemala

Guatemala no cuenta actualmente con una ley general de protección de datos personales equivalente al RGPD europeo. Existe un vacío legal en el ámbito privado que impide a las personas ejercer plenamente sus derechos de acceso, rectificación, cancelación y oposición sobre sus datos. Actualmente existe una iniciativa de ley en el Congreso (Iniciativa 6572, presentada en 2025) que busca crear una Ley de Protección de Datos Personales inspirada en el RGPD, pero aún no ha sido aprobada, por lo que no constituye una obligación legal vigente.

Sin embargo, el proyecto sí debe considerar el marco normativo indirecto existente:

Norma	Relevancia para Ecopac Digital
Constitución Política de la República, Art. 24	Reconoce el derecho a la inviolabilidad de la correspondencia y documentos, base del derecho a la intimidad.
Ley de Acceso a la Información Pública (Decreto 57-2008)	Define legalmente "datos personales" y "datos sensibles" (incluye datos de salud); aunque orientada al sector público, sirve como referencia técnica de buenas prácticas.
Código de Salud (Decreto 90-97)	Establece la confidencialidad del expediente clínico y de la información médica del paciente como obligación del personal de salud.
Código Penal, Art. 223	Tipifica el delito de revelación de secreto profesional, aplicable al personal médico que divulgue información clínica sin autorización.

Implicación de diseño

Dado que no existe una ley específica que imponga un estándar técnico obligatorio, el equipo adopta voluntariamente principios internacionales como: minimización de datos, limitación de finalidad, confidencialidad, integridad, esto como estándar de facto. Esto protege a los

pacientes y deja al sistema mejor posicionado ante una eventual aprobación de la ley pendiente.

Consideraciones Específicas por el Tipo de Datos que Maneja Ecopac

- Datos de salud de pacientes, incluyendo menores de edad sin DPI, identificados únicamente por nombre y comunidad requiere mecanismos de búsqueda/desambiguación que no expongan datos innecesarios.
- Medicamentos de uso controlado ansiolíticos, antiepilépticos, antidepresivos exige trazabilidad estricta de entrega y control de acceso reforzado sobre esos registros de inventario.
- Pacientes con enfermedades crónicas, renales, diabéticos un historial longitudinal de alto valor y sensibilidad si se filtra o se pierde.

Controles de Seguridad Informática

Control de acceso y autenticación

- Autenticación mediante usuario y contraseña, con almacenamiento de contraseñas mediante hashing (bcrypt/argon2), nunca en texto plano.
- Control de acceso basado en roles (RBAC): Administrador, Junta Directiva, Médicos/Voluntarios, cada uno con el mínimo privilegio necesario.
- Acceso a diagnósticos e historiales clínicos restringido exclusivamente a personal médico autorizado y al administrador.
- Bloqueo de cuenta tras intentos fallidos repetidos y expiración de sesión por inactividad, relevante en dispositivos móviles compartidos durante jornadas.

Protección de datos en tránsito y en reposo

- Cifrado de toda comunicación cliente-servidor mediante HTTPS/TLS.
- Cifrado de campos especialmente sensibles en base de datos diagnósticos, medicamentos controlados, además del cifrado estándar de PostgreSQL en reposo.

Integridad y trazabilidad

- Registro de auditoría (logs) de accesos, modificaciones y eliminaciones sobre expedientes clínicos e inventario.

- El flujo ya definido de "movimientos pendientes de validación" en inventario actúa como control de integridad adicional, evitando alteraciones directas sin aprobación del administrador.

Disponibilidad y continuidad

- Respaldos (backups) periódicos y automatizados de la base de datos, con al menos una copia fuera del servidor principal.
- Plan básico de recuperación ante pérdida de datos, adaptado a que Ecopac no cuenta con equipo de TI dedicado.

Estándares Internacionales como Marco de Referencia

No se busca una certificación real fuera de alcance para una ONG sin presupuesto ni auditoría externa sino usar estos marcos como checklist de madurez en seguridad:

- **ISO/IEC 27001 (SGSI):** se adoptan sus principios básicos identificación de activos críticos, expedientes clínicos, inventario, credenciales, análisis simple de riesgos, controles proporcionales, y mejora continua ante el cambio de personal rotativo sin implementar el sistema de gestión completo.
- **ISO/IEC 27799:** extensión de ISO 27001 específica para salud; sirve como referencia conceptual para justificar decisiones ya tomadas, como la restricción de acceso a diagnósticos solo a personal médico y la trazabilidad de accesos al expediente.
- **NIST Cybersecurity Framework:** útil como forma de organizar y presentar los controles ya definidos bajo sus cinco funciones (Identificar, Proteger, Detectar, Responder, Recuperar), mostrando cobertura de todo el ciclo de seguridad y no solo prevención.

Aspectos Organizacionales

- Entregar como parte del proyecto una política mínima de manejo de datos (quién puede ver qué, cuánto tiempo se conservan los datos, procedimiento ante una fuga de información), dado que Ecopac no cuenta con una política escrita y la sostenibilidad del sistema depende de que la organización sepa operarlo sin soporte técnico externo constante.

- Capacitación básica a médicos y voluntarios rotativos sobre manejo responsable de credenciales, dado el riesgo que implica compartir accesos por la rotación frecuente de personal.

OWASP: Seguridad en el Desarrollo de la Aplicación

Dado que el stack definitivo del proyecto es React + Vite + Bootstrap (web), React Native + Expo (móvil), y Supabase (Backend-as-a-Service sobre PostgreSQL), el análisis OWASP se adapta a una arquitectura donde no existe un backend propio tipo Django, sino que la lógica de acceso a datos recae principalmente en Supabase (autenticación, base de datos, políticas de seguridad y almacenamiento). Esto cambia dónde se ubican los controles: gran parte de la seguridad ya no se implementa en "endpoints" propios, sino en la configuración de Supabase.

OWASP Top 10 (Web Application Security Risks)

Riesgo OWASP	Aplicación concreta en Ecopac Digital (React + Vite + Supabase)
A01 – Control de acceso roto (Broken Access Control)	En Supabase, el control de acceso real se implementa mediante Row Level Security (RLS) en PostgreSQL, no confiando únicamente en la lógica del frontend en React. Se deben definir políticas RLS por tabla (patients, medical_records, inventory, journeys) que verifiquen el rol del usuario autenticado (auth.uid() y una tabla de roles) antes de permitir SELECT, INSERT, UPDATE o DELETE. Un Médico/Voluntario nunca debe poder leer o modificar datos fuera de su jornada activa, y esto debe garantizarse a nivel de base de datos, no solo ocultando botones en la interfaz.

A02 – Fallos criptográficos	Supabase fuerza TLS por defecto en todas las conexiones (API REST y conexión directa a PostgreSQL), por lo que no requiere configuración adicional. El punto crítico es la gestión de llaves: la anon key es pública por diseño y puede vivir en el cliente (web y móvil), pero la service_role key (que ignora RLS) jamás debe incluirse en el código de React, Vite o Expo, ni en variables de entorno con prefijo VITE(estas se empaquetan y quedan visibles en el bundle del cliente).
A03 – Inyección (SQL, etc.)	El cliente supabase-js parametriza las consultas mediante PostgREST, mitigando inyección SQL en el uso estándar. El riesgo se traslada a las funciones RPC personalizadas (PL/pgSQL) que se creen en Supabase para lógica compleja (ej. validar stock antes de una entrega); estas deben evitar concatenación dinámica de SQL.
A04 – Diseño inseguro	Diseñar desde el inicio las políticas RLS y las restricciones de base de datos constraints, triggers para reglas críticas como "no entregar medicamentos vencidos" o "movimientos de inventario en estado pendiente hasta aprobación", de forma que la regla exista en la base de datos y no dependa de que el frontend en React la respete correctamente.
A05 – Configuración incorrecta de seguridad	Revisar que RLS esté habilitado en todas las tablas (Supabase permite crear tablas sin RLS activado, lo cual expone los datos completamente si se usa la anon key). Configurar CORS en Supabase para restringir orígenes permitidos, y evitar exponer buckets de Storage (ej. fotos de fichas o evidencias) como públicos si contienen información sensible.
A06 – Componentes vulnerables y desactualizados	– Auditar dependencias regularmente con npm audit para React, Vite y Bootstrap, y con expo doctor / npm audit en el proyecto de Expo. – Mantener actualizado el SDK de Expo, ya que versiones antiguas dejan de recibir soporte y pueden bloquear builds o exponer vulnerabilidades conocidas.

A07 – Fallos de identificación y autenticación	Usar Supabase Auth (email/password) con políticas de contraseña mínima, y aprovechar el manejo nativo de tokens JWT con expiración y <i>refresh tokens</i> . Configurar límites de intentos de inicio de sesión (rate limiting) disponibles en la configuración de Auth de Supabase.
A08 – Fallos de integridad de software y datos	La validación de reglas críticas (stock disponible, medicamento no vencido) debe reforzarse con constraints y triggers en PostgreSQL, no solo con validación en el formulario de React/React Native, ya que un cliente modificado podría saltarse la validación del frontend pero no la de la base de datos.
A09 – Fallos de registro y monitoreo (Logging)	Habilitar y revisar los logs de Supabase (API logs, Auth logs, Database logs) para detectar accesos anómalos. Complementar con una tabla de auditoría propia (audit_log) que registre accesos y cambios sobre expedientes clínicos e inventario, ya que los logs nativos de Supabase pueden tener retención limitada en el plan gratuito.
A10 – Server-Side Request Forgery (SSRF)	Riesgo bajo dado que no hay backend propio haciendo llamadas salientes, pero aplica si se usan Supabase Edge Functions que consuman APIs externas (ej. notificaciones); en ese caso, validar y restringir cualquier URL de destino.

OWASP MASVS (Mobile Application Security Verification Standard)

Aplicado a la app móvil desarrollada en React Native + Expo, que es el canal principal de registro durante las jornadas y el que opera en modo offline:

- Comunicación segura: Supabase fuerza TLS por defecto; verificar que la app no tenga configuraciones de Expo que permitan tráfico HTTP sin cifrar en desarrollo y que esa configuración no se filtre a producción.

- Autenticación y gestión de sesión: aprovechar el manejo de sesión de supabase-js (auto-refresh de JWT) y forzar cierre de sesión tras inactividad prolongada, dado que los dispositivos móviles son compartidos entre personal médico y voluntario rotativo.
- Gestión de la llave pública (anon key): a diferencia de un backend propio, en Supabase es esperado que la anon key esté embebida en el binario de la app. La seguridad real no depende de ocultar esta llave, sino de que las políticas RLS en PostgreSQL sean correctas y restrictivas este es el punto de mayor riesgo si se pasa por alto en el desarrollo con Supabase.
- Resiliencia ante manipulación: no incluir la service_role key de Supabase en ningún build de la app (web ni móvil), ya que esta sí otorga acceso total sin restricciones de RLS.